



Cairo University

Faculty of Economics and Political Science

Department of Statistics

Statistical Compression of VGG19 for Blood-Cell Image Classification on BloodMNIST

A paper submitted in partial fulfillment of the requirements for the degree of Bachelor's in
Economics and Political Science

by

Abdulrahman Mostafa Kamel Hassan Mohamed

Ezz Eldin Ahmed Mohamed Fatouh

Masty Ahmed Saied Said ElHenawy

Mohamed Amir Mohamed Tawfik ElSamahy

Supervised by

Dr. Niveen ElZayaat

May 2026

Acknowledgements

Our deepest gratitude goes to Prof. Fatma El-Zanaty. Sincere appreciation is also extended to Prof. Abdel Nasser Saad for his continuous support and true belief in us. A very special and profound thanks is dedicated to our graduation project supervisor, Dr. Niveen ElZayaan, for her endless help, immense support, and for guiding us toward the exact right path in our research. Furthermore, the valuable advice, support, and confidence shown by Dr. Sara Osama were highly appreciated. Finally, much gratitude belongs to our Teaching Assistant, Mennat Allah Gamal, for her great assistance, as well as to everyone whose inspiration contributed to the completion of this work.

Abstract

This study examines whether statistically grounded compression methods can reduce the complexity of an over-parameterized deep neural network while preserving diagnostically meaningful performance. Using VGG19 fine-tuned on the BloodMNIST+ hematological imaging dataset as a unified testbed, the study evaluates three complementary compression strategies: L_1 Lasso regularization with iterative magnitude pruning, truncated singular value decomposition (SVD), and structured L_0 Gaussian gate regularization followed by post-training tensor surgery. The analysis treats compression as a statistical model-selection problem rather than as a purely heuristic engineering exercise and explicitly distinguishes theoretical sparsity, physical model-size reduction, and realized hardware acceleration.

The results show that the three methods solve different parts of the deployment problem. L_1 Lasso exposed substantial weight-level redundancy, reaching 92.8% unstructured sparsity and 98.29% test macro-F1, a reduction of only 0.28 percentage points from the uncompressed baseline, but it did not improve inference efficiency on standard dense hardware. Truncated SVD provided the strongest observed accuracy-preserving compression result, reaching 98.77% macro-F1 while reducing the physical model size to 82.4 MB, although its latency gains remained modest. Structured L_0 pruning produced the strongest observed deployment outcome after tensor surgery: the deployable model shrank to 134.88 MB and reduced inference latency by 50.5%, while retaining 93.11% macro-F1 and 99.68% macro ROC-AUC.

Taken together, these findings suggest that no single compression method is universally optimal. In these experiments, SVD appears most suitable when predictive fidelity and storage reduction are the main priorities, whereas structured L_0 pruning with tensor surgery appears most suitable when wall-clock efficiency is the main deployment constraint. The study therefore contributes a statistical interpretation of neural network compression, a reproducible set of optimization schedules and guardrails, and a practical decision framework for evaluating compressed diagnostic-imaging models in resource-constrained research settings.

Contents

Acknowledgements	i
Abstract	ii
Abbreviations and Glossary	vii
1 Introduction	2
1.1 Problem Statement and Research Gap	2
1.2 Literature Review	3
1.2.1 The Intersection of Applied Statistics and Deep Learning	3
1.2.2 Over-Parameterization and Benign Overfitting	4
1.2.3 The VGG19–BloodMNIST Setting and Statistical Tuning	4
1.2.4 Deep Neural Network Compression Techniques	5
1.2.5 Statistical and Hessian-Based Pruning	5
1.2.6 Dimensionality Reduction: PCA and SVD	6
1.2.7 Sparse Regularization and Continuous Relaxations	6
1.2.8 Literature Review Summary	7
2 Research Methodology	9
2.1 Statistical Formulation of Neural Networks	9
2.1.1 Neural Networks as Non-Parametric Regression Models	9
2.1.2 Layer-Wise Decomposition (Recursive GLMs)	9
2.1.3 Empirical Risk and Loss Function	10
2.2 Dataset Description and Exploratory Data Analysis	11
2.3 Second-Order Pruning as a Theoretical Baseline	13
2.4 Scalable Statistical Compression Methods	14
2.4.1 Sparse Regularization via L_1 Norm (Lasso)	14
2.4.2 Differentiable L_0 Regularization and Gaussian Gates	15
2.4.3 Dimensionality Reduction via Singular Value Decomposition (SVD)	15
2.5 Generalization and Model Complexity Motivation	16
3 Baseline Model Analysis	19
3.1 Introduction	19
3.2 Architectural Analysis of VGG19	19
3.3 Fine-Tuning Methodology	20
3.4 Empirical Performance and Class-Level Behavior	21
3.4.1 Macro-Level Statistical Metrics	21
3.4.2 Per-Class Evaluation	21
3.5 Computational Complexity Assessment	24
3.6 Baseline Summary	24
4 Compression Results and Structural Analysis	26
4.1 L1 Lasso Regularization	26
4.1.1 L1 Rationale and Training Configuration	26
4.1.2 Training Dynamics and Sparsity Transition	26
4.1.3 Iterative Magnitude Pruning (IMP) and Structural Refinement	28

4.1.4	Empirical Evaluation and Class-Level Behavior	28
4.1.5	Statistical Efficiency vs. Hardware Limitations	30
4.2	Dimensionality Reduction via Singular Value Decomposition (SVD)	31
4.2.1	Theoretical Rationale and Experimental Configuration	31
4.2.2	Energy Threshold Sweep and Rank Selection	31
4.2.3	Fine-Tuning Dynamics	31
4.2.4	Empirical Evaluation and Class-Level Behavior	32
4.2.5	Statistical Efficiency vs. Hardware Limitations	32
4.3	Structural Compression via L0 Gaussian Gating and Tensor Surgery	34
4.3.1	Empirical Justification for Structural Compression (PCA Audit)	34
4.3.2	Architectural Design: L0 Gaussian Gates	34
4.3.3	Training and PID-Controlled Sparsification	35
4.3.4	Masked Sparsity and Physical Tensor Surgery	37
4.3.5	Diagnostic Performance After Structured Pruning	38
4.3.6	Hardware Deployment Metrics	38
5	Discussion and Unified Methodological Comparison	42
5.1	Comparison of Compression Methods	42
5.2	Hardware Efficiency and Structural Compression	44
5.3	Class-Level and Architectural Interpretations	44
6	Conclusions and Recommendations	48
6.1	Summary of Key Findings	48
6.2	Methodological Contributions	48
6.3	Recommendations for Practitioners	49
6.4	Recommendations for Future Research	49
6.5	Conclusion	50
	References	51
A	Appendix A: Hardware and Software Environment Specifications	55
A.1	Hardware Infrastructure	55
A.2	Software, AI Tools, and Libraries	55
A.3	Reproducibility Controls	56
B	Appendix B: Hyperparameter Configurations and Optimization Schedules	58
B.1	Hyperparameter Selection Rationale	58
B.2	Baseline VGG19 Training	59
B.3	L_1 Lasso Regularization and IMP	59
B.4	Singular Value Decomposition (SVD)	62
B.5	L_0 Gaussian Gates and PID Controller	63
C	Appendix C: Dense Reconstruction via Tensor Surgery	65
C.1	Tensor Surgery Procedure	65
C.2	Numerical Consistency Check	66

List of Figures

1	BloodMNIST class-distribution profile	11
2	RGB pixel-intensity distributions	12
3	Representative BloodMNIST cell samples	13
4	SVD scree plot for fully connected layers	16
5	SVD singular-value spectra	17
6	VGG19 architecture schematic	19
7	Baseline activation-space PCA	20
8	Baseline VGG19 training dynamics	21
9	Baseline confusion matrix	22
10	Baseline multi-class ROC curves	23
11	Baseline class-wise F1-score distribution	23
12	L1 Lasso training dynamics	27
13	Post-L1 weight magnitude distribution	27
14	L1 Lasso confusion matrix	29
15	Baseline and L1 per-class F1-score comparison	29
16	Post-training L0 gate distribution	35
17	L0 Gaussian gate training dynamics	36
18	L0 confusion matrix	38
19	Unified Pareto comparison	42
20	Grouped predictive-metric comparison	43
21	AIC and BIC comparison across models	45
22	Per-class F1-score heatmap	46

List of Tables

1	BloodMNIST class distribution	12
2	SVD rank-cap diagnostic for fully connected layers	16
3	Baseline class-wise diagnostic performance	22
4	L1 pruning and refinement results	28
5	Baseline vs L1 class-wise comparison	28
6	SVD energy threshold sweep	31
7	Baseline vs SVD class-wise comparison	32
8	PCA redundancy audit	34
9	Per-layer L0 gate survival	37
10	L0 Gaussian gate results summary	39
11	L0 per-class performance vs baseline	40
12	Unified performance metrics	43
13	Structural compression and efficiency metrics	43
14	Information-theoretic metrics	44
15	Hardware environment	55
16	Software environment	55
17	AI tools used	56
18	Primary Python libraries	56
19	Baseline optimization parameters	59
20	Baseline VGG19 training dynamics	59
21	L_1 Lasso optimization parameters	60
22	L_1 training dynamics, epochs 1–22	60
23	L_1 training dynamics, continued	61
24	IMP refinement schedule	61
25	IMP and masked refinement	61
26	SVD factorization parameters	62
27	SVD fine-tuning dynamics	62
28	L_0 gate optimization parameters	63
29	L_0 PID training dynamics, epochs 1–10	63
30	L_0 PID training dynamics, epochs 11–20	64
31	L_0 PID training dynamics, epochs 21–30	64
32	L_0 PID training dynamics, epochs 31–40	64
33	PID sparsity controller dynamics	64

Abbreviations and Glossary

Abbreviations

Abbreviation	Full Term	Definition
AI	Artificial Intelligence	Computational systems performing tasks that ordinarily require human cognitive ability, including pattern recognition and classification. (Goodfellow et al., 2016; Min et al., 2024).
Adam	Adaptive Moment Estimation	Gradient-based optimizer that combines momentum and adaptive learning-rate scaling; used here for SVD fine-tuning and L_0 Gaussian gate training. (Kingma & Ba, 2015).
AUC	Area Under the ROC Curve	Threshold-independent measure of class separability; a value of 1.0 denotes perfect discrimination. (Fawcett, 2006).
BLAS	Basic Linear Algebra Sub-programs	Low-level matrix computation routines optimised for dense operands; provide no acceleration for zero-valued weight entries unless sparse BLAS kernels are explicitly invoked. (Lawson et al., 1979).
BloodMNIST+	Blood-cell MNIST Dataset	High-resolution benchmark of 17,092 peripheral blood-cell images across eight haematological categories, drawn from the MedMNIST family. (Yang et al., 2023).
CNN	Convolutional Neural Network	Neural network architecture in which successive layers apply learned spatial filters, progressively abstracting local patterns into morphological features. (Goodfellow et al., 2016).
DNN	Deep Neural Network	A neural network with multiple hidden layers capable of approximating highly non-linear functions. (Goodfellow et al., 2016).
FC	Fully Connected Layer	Layer in which every output neuron receives a weighted combination of all input neurons; equivalent in form to a standard linear predictor. (Goodfellow et al., 2016).
FLOPs	Floating-Point Operations	Total arithmetic operations per forward pass; hardware-agnostic measure of computational cost. (Francy & Singh, 2024; Liu et al., 2025).
F1-score	F1 Score	Harmonic mean of Precision and Recall. Macro F1-score averages across classes with equal weight, making it sensitive to minority-class degradation. (Powers, 2011).
GLM	Generalised Linear Model	Used as a statistical analogy: each network layer combines a linear predictor with a non-linear activation, similar in structure to a generalized linear modelling step. (Breiman, 2001; Goodfellow et al., 2016).
HOS	Higher-Order Statistics	Statistical moments beyond the second order. Referenced here in the context of Chatzikonstantinou et al., who used HOS to identify filters with near-Gaussian weight distributions as candidates for pruning. (Chatzikonstantinou et al., 2020).

Abbreviation	Full Term	Definition
IID	Independent and Identically Distributed	Standard statistical assumption that observations are sampled independently from the same data-generating distribution; used here to distinguish in-distribution Blood-MNIST evaluation from out-of-distribution deployment. (Vapnik & Chervonenkis, 1971; Goodfellow et al., 2016).
IMP	Iterative Magnitude Pruning	Procedure that cycles between zeroing the lowest-magnitude weights and retraining the residual network, progressively increasing hard sparsity. (Frankle & Carbin, 2019; Blalock et al., 2020).
ImageNet	ImageNet Large Scale Visual Recognition Challenge	Repository of over 14 million labelled photographs across 1,000 categories; source of VGG19’s pre-trained weights. (Simonyan & Zisserman, 2015).
L0	L0 Norm	Count of non-zero parameters. Non-differentiable; requires a continuous relaxation during gradient-based optimisation. (Louizos, Welling, & Kingma, 2018).
L1 / Lasso	L1 Norm / Least Absolute Shrinkage and Selection Operator	Regularisation penalty corresponding to a Laplace prior; shrinks small weights toward zero and supports later magnitude pruning. (Tibshirani, 1996).
L2 / Ridge	L2 Norm / Ridge Regularisation	Regularisation penalty corresponding to a Gaussian prior; shrinks weights continuously without producing exact zeros. (Goodfellow et al., 2016).
LR	Likelihood-Ratio Contrast	Signed log-likelihood contrast against the uncompressed baseline, used to summarize relative model fit alongside AIC and BIC. (Burnham & Anderson, 2002).
MCC	Matthews Correlation Coefficient	Pearson correlation between predicted and observed class labels across all classes; robust to class imbalance. (Matthews, 1975).
MDL	Minimum Description Length	Model-selection principle favouring the hypothesis that minimises the combined description length of the model and the residuals it leaves unexplained. (Rissanen, 1978).
MedMNIST	Medical MNIST Benchmark	Standardised collection of lightweight medical-imaging datasets designed for reproducible benchmarking. (Yang et al., 2023).
OBD	Optimal Brain Damage	Second-order pruning framework (LeCun et al., 1990) using a diagonal Hessian approximation to identify parameters whose removal incurs the smallest training loss increase.
OBS	Optimal Brain Surgeon	Extension of OBD (Hassibi & Stork, 1993) incorporating the full inverse Hessian to account for parameter correlations and compute compensatory weight adjustments.
OOD	Out-of-Distribution	Refers to inputs whose data distribution differs from the training distribution, for which predictive reliability cannot be assumed a priori. (Goodfellow et al., 2016; Nalisnick, Smyth, & Tran, 2023).
OvR	One-vs-Rest	Multi-class evaluation strategy in which each class is treated as positive against all others; used here to compute per-class ROC curves. (Fawcett, 2006).

Abbreviation	Full Term	Definition
PCA	Principal Component Analysis	Used in its standard sense. Applied here to audit the intrinsic dimensionality of each layer’s activation space, providing empirical justification for SVD compression. (Jolliffe, 2002).
PID	Proportional–Integral–Derivative Controller	Feedback mechanism used here to modulate the sparsity penalty dynamically during L0 training, driving the network toward a 60% gate-sparsity target. (Åström & Hägglund, 1995).
ReLU	Rectified Linear Unit	Activation function defined as $\max(0, x)$; serves as the non-linear link function between successive network layers. (Goodfellow et al., 2016).
ROC	Receiver Operating Characteristic	Standard diagnostic curve plotting True Positive Rate against False Positive Rate across classification thresholds. (Fawcett, 2006).
SGD	Stochastic Gradient Descent	Iterative optimisation in which the gradient is approximated from a random mini-batch at each step rather than the full training set. (Goodfellow et al., 2016).
SGDR	SGD with Warm Restarts	SGD variant with periodic learning-rate resets; employed in L1 training to prevent premature convergence near the pruning threshold. (Loshchilov & Hutter, 2017).
SVD	Singular Value Decomposition	Standard matrix factorisation $W = U\Sigma V^T$. Truncating the smallest singular values yields the optimal low-rank approximation by the Eckart–Young–Mirsky theorem. (Eckart & Young, 1936; Mirsky, 1960).
VGG19	Visual Geometry Group Network (19 layers)	Deep CNN with 19 weight layers and approximately 140 million parameters (Simonyan & Zisserman, 2014), pre-trained on ImageNet. (Simonyan & Zisserman, 2015).
VRAM	Video Random Access Memory	Dedicated GPU memory holding weight tensors, activation maps, and gradient buffers during training and inference. (Francy & Singh, 2024).

Glossary

Term	Definition
Benign overfitting	Perfect interpolation of training data alongside strong generalisation to unseen observations, inconsistent with classical bias–variance theory and attributed to the implicit regularisation effect of SGD in over-parameterised networks. (Bartlett et al., 2021; Nalisnick, Smyth, & Tran, 2023).
Checkpoint selection	Retention of the model state that maximises weight sparsity subject to validation F1 remaining within five percentage points of the observed peak. (Goodfellow et al., 2016).
Compression	Reduction in model size, parameter count, or computational burden while preserving acceptable predictive performance. (Blalock et al., 2020; Liu et al., 2025).
Dark knowledge	The information encoded in a trained model’s probability assignments to non-predicted classes, reflecting learned inter-class similarity. Proposed as a target signal for future knowledge distillation experiments. (Hinton, Vinyals, & Dean, 2015).
Dense hardware	Hardware optimised for dense matrix operations that provides no automatic acceleration for zero-valued weight entries. (Kuzmin et al., 2021; Francy & Singh, 2024).
Dense pruned model	The physically reconstructed network produced by tensor surgery, retaining only surviving channels and executable on standard hardware. (Blalock et al., 2020; Francy & Singh, 2024).
Depth-dependent pruning gradient	The pattern whereby early convolutional layers retain more channels than deeper layers after structured pruning, reflecting the importance of low-level visual features for haematological classification. (Blalock et al., 2020; Acharyya et al., 2021).
Diagnostic performance	Predictive effectiveness measured by accuracy, macro F1-score, ROC-AUC, and MCC. (Powers, 2011; Fawcett, 2006; Matthews, 1975).
Double descent	Empirical relationship in which test error decreases, rises near the interpolation threshold, then decreases again in the heavily over-parameterised regime; theoretical basis for training large networks and compressing them post-hoc. (Bartlett et al., 2021; Nalisnick, Smyth, & Tran, 2023).
Eckart–Young–Mirsky theorem	The best rank- k matrix approximation in Frobenius norm is obtained by retaining the k largest singular values; the theoretical guarantee underpinning truncated SVD compression. (Eckart & Young, 1936; Mirsky, 1960).
Energy threshold	Minimum proportion of singular-value variance retained after SVD truncation; set to 0.20 in this study. (Eckart & Young, 1936; Mirsky, 1960).
Fine-tuning	Continued gradient-based training of a pre-trained model on a smaller target dataset. (Goodfellow et al., 2016; Simonyan & Zisserman, 2015).
Gate survival	Fraction of convolutional channels whose L0 gate parameter remains active following training; primary index of retained channel capacity. (Louizos, Welling, & Kingma, 2018).
Gaussian gate regularisation	Continuous stochastic gates parameterised by a trainable location and noise scale, initialized near the open/closed decision boundary to permit progressive channel closure without destabilising pre-trained weights. (Louizos, Welling, & Kingma, 2018; Oliveira, Batista, & Seara, 2023).
Ghost sparsity	High theoretical sparsity with no improvement in on-disk size or inference latency, because the hardware continues to process zero-valued entries. Resolved by tensor surgery. (Kuzmin et al., 2021; Francy & Singh, 2024).

Term	Definition
Hard Concrete distribution	An L0 gate relaxation (Louizos et al., 2018) placing probability mass on exact zero and one states; used here as the theoretical reference point for the later Gaussian gate relaxation. (Louizos, Welling, & Kingma, 2018).
Knowledge distillation	A compression technique in which a compact student network is trained to reproduce the full output probability distributions of a larger teacher. Identified as a direction for future work. (Hinton, Vinyals, & Dean, 2015; Gou et al., 2021).
Low-rank factorisation	Representation of a weight matrix as a product of two smaller matrices, reducing both storage and the number of multiply-accumulate operations. (Eckart & Young, 1936; Mirsky, 1960).
Masked model	Intermediate model state in which pruned channels are zeroed during the forward pass but remain physically in memory, producing ghost sparsity prior to tensor surgery. (Louizos, Welling, & Kingma, 2018; Blalock et al., 2020).
Over-parameterisation	Substantially more trainable parameters than training observations; present in VGG19 at a parameter-to-sample ratio exceeding 11,600:1 on BloodM-NIST. (Bartlett et al., 2021; Nalisnick, Smyth, & Tran, 2023).
Pareto frontier	Set of solutions where no objective can be improved without degrading another; used to position each compression method within the accuracy–size–latency trade-off space. (Burnham & Anderson, 2002; Francy & Singh, 2024).
Pruning	The systematic removal of parameters, channels, or structural units from a trained neural network on the basis of a saliency criterion, with the objective of reducing model size and computational cost while preserving predictive performance. Pruning may operate at the weight level (unstructured) or at the channel and block level (structured), with the latter being necessary for realised hardware acceleration. (LeCun et al., 1990; Blalock et al., 2020).
Phase transition	Abrupt degradation in validation performance when accumulated regularisation pressure exceeds the network’s remaining representational capacity; observed here during L1 training. (Bartlett et al., 2021; Goodfellow et al., 2016).
Physical compression	Realised reduction in on-disk model size after inactive weights, channels, or matrix factors have been removed from the stored architecture. (Blalock et al., 2020; Francy & Singh, 2024).
Shrinkage bias	Systematic underestimation of large weights by L1 regularisation, which applies an equal gradient penalty to all parameters regardless of magnitude. (Tibshirani, 1996).
Singular value spectrum	The ordered sequence $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r$ from SVD; its rapid decay in VGG19’s FC layers provides empirical justification for truncated SVD compression. (Eckart & Young, 1936; Mirsky, 1960).
Soft labels	The full output probability vector produced by a trained model across all classes, as opposed to a single hard-class prediction; central to knowledge distillation. (Hinton, Vinyals, & Dean, 2015).
Structured pruning	Removal of entire channels or blocks, producing architectures compatible with standard hardware and yielding real reductions in inference latency. (Blalock et al., 2020; Acharyya et al., 2021).
Tensor surgery	Post-training algorithm that identifies surviving channel indices, physically slices weight tensors, and re-maps FC input dimensions to produce a smaller deployable network. (Blalock et al., 2020; Francy & Singh, 2024).

Term	Definition
Theoretical sparsity	Proportion of parameters set to zero, irrespective of whether storage or runtime improve in practice. (Kuzmin et al., 2021; Blalock et al., 2020).
Transfer learning	Initialisation with weights pre-trained on ImageNet prior to adaptation to BloodMNIST. (Goodfellow et al., 2016; Simonyan & Zisserman, 2015).
Unstructured sparsity	Weight-level zeroing that does not alter matrix dimensions and therefore yields no latency improvement on dense hardware. (Blalock et al., 2020; Kuzmin et al., 2021).
Wall-clock efficiency	Actual elapsed inference time in milliseconds per batch, as distinct from theoretical FLOPs. (Francy & Singh, 2024).
Zero-attraction effect	The constant gradient force of L1 regularisation that drives small-magnitude weights toward zero, analogous to a Laplace prior in a Bayesian formulation. (Tibshirani, 1996).

Chapter 1

Introduction

1 Introduction

Artificial intelligence (AI), especially deep learning (DL), is now used in many areas of computing. From a statistical view, a deep neural network is a layered function estimator trained to reduce prediction error on observed data (Goodfellow, Bengio, & Courville, 2016; Bartlett et al., 2021). Much of its recent success comes from scale: large models can learn complex patterns, but their flexibility also increases model complexity.

This predictive power has a high computational cost. VGG19, introduced by Simonyan and Zisserman (2015), contains about 140 million parameters. It requires hundreds of megabytes of storage and many floating-point operations to classify one image. The cost is even larger for modern foundation models, which often need specialized hardware for inference (Kaplan et al., 2020).

Because of this, large neural networks are difficult to use on systems with limited resources. These systems include mobile devices, embedded platforms, and point-of-care medical hardware. In these settings, memory, speed, energy use, and latency are practical constraints. Without compression, many deep models are too large or too slow to use outside high-performance hardware (Howard et al., 2017; Tan & Le, 2019; Francy & Singh, 2024; Liu et al., 2025).

Model compression is one way to address this problem. Common methods include pruning, quantization, knowledge distillation, and low-rank factorization (Han, Mao, & Dally, 2016; Blalock et al., 2020; Hinton, Vinyals, & Dean, 2015; Gou et al., 2021; Liu et al., 2025). These methods can reduce model size and inference cost while preserving much of the original accuracy. However, many compression methods are still guided mainly by engineering heuristics and hardware-oriented constraints (Kuzmin et al., 2021; Francy & Singh, 2024). They do not always explain which parameters are redundant, how much compression is safe, or whether the smaller model will keep low generalization error on new data.

In this study, we examine model compression from a statistical point of view. Instead of treating a deep neural network only as an engineering system, we treat it as a flexible statistical estimator with more parameters than the task may need (Breiman, 2001; Bartlett et al., 2021). From this view, removing parameters is a model-selection problem. The study uses statistical ideas such as sparsity, low-rank approximation, regularization, and effective model complexity to ask which parts of the network carry signal and which parts mostly add variance or computational cost.

1.1 Problem Statement and Research Gap

Although model compression is widely studied, a gap remains between making networks smaller and explaining compression as model selection (Blalock et al., 2020; Liu et al., 2025). Many studies report accuracy, sparsity, or parameter reduction. Fewer compare statistical compression methods in the same medical-imaging pipeline while separating three different outcomes: sparsity on paper, actual parameter reduction, and measured inference speed. This distinction matters because a method can set many weights to zero without making the stored model smaller or faster on dense hardware (Kuzmin et al., 2021). In medical-image classification, compression must also preserve more than overall accuracy; it must maintain class-level performance, especially for minority and

visually similar classes, and should not be confused with clinical readiness without external validation (Kelly et al., 2019).

The problem addressed in this study is therefore the lack of a unified statistical comparison of compression methods for an over-parameterized convolutional network in a controlled BloodMNIST setting. The study asks whether VGG19 has redundant capacity, whether that redundancy can be estimated and reduced using statistical compression tools, and whether the smaller models keep enough predictive performance for practical deployment.

The main research questions are:

1. To what extent can L_1 Lasso regularization, L_0 Gaussian gate regularization, and truncated SVD reduce the complexity of a VGG19 model trained on BloodMNIST?
2. How much predictive performance remains after compression when measured by accuracy, macro-averaged F1-score, and class-level error patterns?
3. Do the three methods produce different forms of redundancy reduction, such as weight shrinkage, learned sparsity, or low-rank structure?
4. How closely do statistical compression measures, such as sparsity or rank reduction, translate into physical model-size reduction and measured inference speed?

The main goal of this study is to test whether statistically motivated compression methods can reduce the size and complexity of an over-parameterized deep network while preserving diagnostic performance. To answer this question, we evaluate three methods on VGG19 trained on BloodMNIST: L_1 Lasso regularization (Tibshirani, 1996), truncated singular value decomposition (Eckart & Young, 1936; Mirsky, 1960), and L_0 Gaussian gate regularization building on differentiable sparsity relaxations (Louizos, Welling, & Kingma, 2018). The study frames compression as statistical model selection and separates three quantities that are often mixed together: theoretical sparsity, physical model size, and real hardware speed.

The rest of this report is organized as follows. Chapter 2 describes the dataset, baseline architecture, evaluation metrics, and mathematical form of the three compression methods. Chapter 3 measures the baseline performance of the uncompressed VGG19 model. Chapter 4 presents the compression results for L_1 Lasso regularization, truncated SVD, and L_0 Gaussian gate regularization. Chapter 5 compares the methods and discusses efficiency, redundancy, and deployment. Chapter 6 summarizes the main contributions, limitations, and future work.

1.2 Literature Review

1.2.1 The Intersection of Applied Statistics and Deep Learning

Machine learning and statistics have often been treated as separate fields: machine learning usually emphasizes prediction accuracy, while statistics often emphasizes data-generating processes,

uncertainty, and interpretation. Breiman’s essay on “statistical modeling: the two cultures” argued that statistics includes both explicit probability modeling and algorithmic modeling, where the prediction rule is learned from data (Breiman, 2001). From this viewpoint, modern deep learning continues the algorithmic modeling culture, where the statistical questions include prediction error, generalization, validation, calibration, and efficient use of data (Guo et al., 2017).

This perspective explains why a statistical study can examine neural-network compression. The aim is to examine a computer-science model using statistical ideas: model complexity, regularization, sparsity, dimensionality reduction, and generalization after parameter removal. A feed-forward neural network can be viewed as a non-linear, non-parametric statistical estimator that learns an unknown pattern from data by minimizing prediction loss (Bartlett et al., 2021).

Recent applied-statistics work also emphasizes data collection, uncertainty measurement, model validation, convergence analysis, and network-design guidance for AI systems (Min et al., 2024; Wu et al., 2025). Overall, this literature suggests that statistics does not replace deep learning practice, but gives it clearer tools for measuring uncertainty, bias, variance, and generalization.

1.2.2 Over-Parameterization and Benign Overfitting

A major feature of modern Convolutional Neural Networks (CNNs) is over-parameterization. These models often have far more parameters than training samples. In a simple degrees-of-freedom view, such models might be expected to overfit and generalize poorly.

Deep learning often behaves differently. Work on double descent and benign overfitting shows that, after a model passes the interpolation threshold, test error can sometimes decrease instead of increase (Bartlett et al., 2021; Nalisnick, Smyth, & Tran, 2023). Stochastic gradient descent (SGD) also appears to act as implicit regularization by guiding the model toward simpler solutions among many possible fits. This helps explain why very large models can be useful during training, even if they later need compression for deployment.

Scaling-law work supports the same practical tension: training may benefit from a large model, but deployment may require a smaller estimator with fewer active parameters (Kaplan et al., 2020).

1.2.3 The VGG19–BloodMNIST Setting and Statistical Tuning

Benign overfitting helps explain why large models can generalize well, but it also creates a practical problem. The final inference model may contain many redundant parameters. This problem is especially important when large image-classification models are transferred to specialized medical tasks (Mohamad et al., 2024).

VGG19 is useful for studying this issue. It was developed for large-scale image recognition by Simonyan and Zisserman (2015), and it contains more than 140 million parameters. Architecturally, VGG19 contains sixteen convolutional layers followed by three fully connected layers, so its capacity is distributed across both the convolutional feature extractor and the dense classifier. When it is

applied to BloodMNIST+, a high-resolution variant of the MedMNIST blood-cell benchmark with eight classes, the model has high representational capacity (Yang et al., 2023). At the same time, it is much larger than the task appears to require. This makes the VGG19–BloodMNIST+ setting a useful case for studying effective degrees of freedom, redundancy, and compression.

Mohamad et al. (2024) describe a related idea called statistical tuning, where variance analysis is used to measure the practical value of hidden units. In this sense, the VGG19–BloodMNIST+ setting is well suited for testing whether statistically guided parameter reduction can preserve useful accuracy while lowering deployment cost.

1.2.4 Deep Neural Network Compression Techniques

Model compression aims to reduce memory use and inference time while keeping test performance stable (Liu et al., 2025). Common compression methods include:

1. **Network pruning** removes weights or larger structures that are less important, creating sparsity in the model (Blalock et al., 2020). Unstructured pruning removes individual weights, while structured pruning removes larger parts such as filters or channels.
2. **Network quantization** reduces the numerical precision of weights and activations. For example, it may replace 32-bit floating-point values with lower-precision values such as 8-bit integers (Wu et al., 2018).
3. **Knowledge distillation** trains a smaller student network to match the outputs of a larger teacher network (Hinton et al., 2015; Gou et al., 2021).

Pruning is especially important for this study because it directly targets redundant parameters. More generally, compression methods affect statistical and hardware quantities differently, so no single method is best for every deployment goal (Kuzmin et al., 2021).

1.2.5 Statistical and Hessian-Based Pruning

Simple pruning rules can remove important parameters and reduce accuracy. For this reason, many studies examine pruning from a statistical perspective. In this view, pruning is an optimization problem: remove parameters with the smallest estimated effect on the loss.

LeCun et al. (1990) developed this idea in the Optimal Brain Damage (OBD) framework, which uses a second-order Taylor approximation and a diagonal Hessian approximation to assign each parameter a saliency score. Parameters with the smallest estimated effect on the loss are pruned first. Hassibi and Stork (1993) extended this idea in the Optimal Brain Surgeon (OBS) framework, which uses the full inverse Hessian to account for parameter relationships and compensating updates.

Other work studies redundancy through the statistical properties of the weights themselves. Chatzikonstantinou et al. (2020) proposed a pruning rule based on higher-order statistics (HOS).

They argue that filters with approximately Gaussian weight distributions may contain less discriminative information. More recent work also treats pruning as structured sparsification rather than only as a simple deletion rule (Acharyya et al., 2021). Post-training methods such as Optimal Brain Compression combine pruning and quantization while trying to preserve accuracy (Frantar & Alistarh, 2022). Together, these studies support the idea that pruning can be guided by statistical saliency measures, not only by simple magnitude thresholds.

1.2.6 Dimensionality Reduction: PCA and SVD

Another way to study redundancy is through dimensionality reduction. Instead of asking which individual parameters should be removed, these methods ask whether activations or weight matrices lie near a lower-dimensional subspace with little information loss.

Principal Component Analysis (PCA) is often used to study activations and identify the main directions of variation (Jolliffe, 2002). If a layer is wider than the task requires, part of its activation space may capture repeated or noisy information rather than useful signal. In such cases, PCA can help identify redundant components. Recent work on sufficient dimension reduction in neural networks also supports the idea that high-dimensional learned features may store task-relevant information in a smaller subspace (Xu & Yu, 2025).

Singular value decomposition (SVD) is a related matrix-level method. If the singular values of a large weight matrix decrease quickly, truncated SVD can approximate the matrix with lower rank. This gives an optimal low-rank approximation under the Eckart–Young–Mirsky theorem (Eckart & Young, 1936; Mirsky, 1960). For this reason, PCA and SVD are useful tools for finding redundancy in activations and parameters, especially in the large fully connected layers of over-parameterized networks.

1.2.7 Sparse Regularization and Continuous Relaxations

Post-training pruning removes parameters after training. Sparse regularization takes a different approach by adding a sparsity penalty during training. Statistically, this changes the objective function so that the model balances prediction loss against model complexity.

The L_1 norm, or Lasso, is a common convex approximation to a sparsity penalty. The original Lasso method used L_1 regularization for both shrinkage and variable selection (Tibshirani, 1996). This makes it useful as a soft sparsity penalty in neural networks. However, L_1 regularization also has a drawback: it can shrink both large and small weights, which may bias important parameters.

For this reason, many studies view the L_0 norm as a cleaner sparsity objective. It counts the number of non-zero parameters instead of penalizing their size. The problem is that the discrete L_0 norm is not differentiable, so it cannot be optimized directly with standard gradient methods. Practical methods therefore use continuous relaxations. Louizos et al. (2018) proposed an important approach based on stochastic hard-concrete gates. Later studies using L_0 regularization and weight pruning show that L_0 methods are best understood as direct controls on sparsity, not as replacements for L_1

shrinkage (Yamamoto & Maeno, 2018; Oliveira, Batista, & Seara, 2023). Across this literature, the goal is to create sparse models while limiting accuracy loss.

The Lottery Ticket Hypothesis gives another reason to expect sparse subnetworks inside large models. It argues that dense networks may contain smaller trainable subnetworks that can match the original model when initialized properly (Frankle & Carbin, 2019). This supports the broader idea that over-parameterized models can contain redundant parts that may be removed without destroying performance.

1.2.8 Literature Review Summary

The literature shows that the deployment challenge of deep neural networks is closely tied to over-parameterization. It also shows that redundancy does not have to be treated only as an engineering problem. Statistical methods based on curvature, dimensionality reduction, and sparse regularization can estimate which parts of a model add signal and which parts mainly add complexity. These ideas motivate Chapter 2, where we formalize compression as statistical model selection for the VGG19–BloodMNIST+ setting.

Chapter 2

Research Methodology

2 Research Methodology

Building on Chapter 1, this chapter turns the statistical ideas of over-parameterization, sparsity, and dimensionality reduction into a practical method. The goal is to treat neural-network compression as statistical model selection: what is removed, why it is reasonable to remove it, and how the smaller model is tested. This motivation is related to the curse of dimensionality (Bellman, 1957), because the relevant dimension includes not only image pixels but also learned parameters and hidden features inside VGG19. Compression is therefore treated as a reduction in effective dimension, keeping directions that carry predictive signal while removing directions that mainly add variance, storage cost, or inference time.

2.1 Statistical Formulation of Neural Networks

2.1.1 Neural Networks as Non-Parametric Regression Models

The equations in this subsection set notation; they do not introduce a new neural-network formulation (Goodfellow, Bengio, & Courville, 2016; Bartlett et al., 2021). In this study, a neural network is treated as a flexible, non-parametric regression or classification estimator. Unlike linear regression, where the form $Y = X\beta + \epsilon$ is fixed in advance, a neural network learns a more flexible function from the data.

A useful statistical way to read this formulation is to view each layer as a learned transformation of the previous representation. The input layer contains the observed covariates, such as image pixels or feature maps. The weights play the same structural role as coefficients, while the bias is the intercept term: together, they determine how information is combined before it is passed forward. The activation function then controls whether the transformation remains linear or becomes non-linear. A model with only one linear transformation and an identity activation reduces to an ordinary linear predictor, while a deep network obtains its flexibility by stacking many such transformations with non-linear activations. After training, prediction on a new image is simply a forward evaluation of the learned function; the parameters are not updated at inference time.

We define the relationship between the input vector X and the response vector Y as:

$$Y = f(X; \theta) + \epsilon \tag{2.1}$$

where $f(\cdot)$ is the non-linear function estimated by the network, $\theta = \{W, b\}$ is the set of learnable weights and biases, and ϵ is the random error term for variation not explained by the model.

2.1.2 Layer-Wise Decomposition (Recursive GLMs)

To connect deep learning with familiar statistical notation, we write $f(X; \theta)$ layer by layer. A deep neural network is a sequence of linear transformations followed by non-linear activation functions.

For a network with L layers, the output of layer l , denoted $h^{(l)}$, is:

$$h^{(l)} = \sigma(W^{(l)}h^{(l-1)} + b^{(l)}) \quad (2.2)$$

where $W^{(l)}h^{(l-1)} + b^{(l)}$ is the linear part of the layer. It plays a role similar to the linear predictor $\eta = X\beta$ in a generalized linear model. The function $\sigma(\cdot)$, such as ReLU, then transforms this linear predictor before passing it to the next layer.

In a convolutional network such as VGG19, convolutional layers apply small learned filters across local image regions. These layers extract spatial patterns such as edges, textures, shapes, and increasingly complex cell structures while sharing the same filter weights across locations. Fully connected layers play a different role. After the convolutional feature maps have been flattened or pooled into a feature vector, a fully connected layer combines all incoming features through a dense weight matrix. In statistical terms, it acts like a high-dimensional learned linear predictor followed by an activation function, allowing the classifier to combine the extracted image features into class-specific evidence. This is why the fully connected head often contains many parameters and is a natural target for low-rank compression through SVD.

This notation lets us treat compression as parameter reduction. The objective is to keep the predictive behavior of $f(X; \theta)$ while reducing the effective dimension of θ .

2.1.3 Empirical Risk and Loss Function

To estimate θ , training minimizes empirical risk over the observed sample $\{(x_i, y_i)\}_{i=1}^n$. In general form, the optimization problem is

$$\hat{\mathcal{L}}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(x_i; \theta)), \quad (2.3)$$

where $\ell(\cdot)$ is the loss for one observation. For multi-class image classification, the network output is passed through a softmax layer to produce estimated class probabilities

$$\hat{p}_{ik} = \frac{\exp(a_{ik})}{\sum_{j=1}^K \exp(a_{ij})}, \quad (2.4)$$

where a_{ik} is the logit for observation i and class k , and K is the number of diagnostic classes.

The corresponding loss is categorical cross-entropy, which is the negative log-likelihood of a multinomial model:

$$\hat{\mathcal{L}}_{\text{CE}}(\theta) = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log \hat{p}_{ik}. \quad (2.5)$$

Here, $y_{ik} \in \{0, 1\}$ is the dummy variable indicating whether observation i belongs to class k . Cross-entropy gives a large penalty to confident wrong predictions, which is important in diagnostic image analysis. In this study, cross-entropy is the data-fit term. The compression methods then add penalties or constraints for sparsity or low-rank structure.

2.2 Dataset Description and Exploratory Data Analysis

For the empirical analysis, we use the BloodMNIST dataset from the MedMNIST benchmark family (Yang et al., 2023). The higher-resolution BloodMNIST+ images are resized to 224×224 pixels to match the VGG19 input size. The dataset contains 17,092 labelled peripheral blood-cell images across eight classes, split into training, validation, and blind test sets of 11,959, 1,712, and 3,421 images.

It should be noted that the BloodMNIST dataset is intended for research and benchmarking purposes only. The models developed in this study are NOT intended for clinical diagnosis or medical decision-making, consistent with the broader caution that medical AI systems require external validation, workflow integration, and clinical governance before deployment (Kelly et al., 2019).

The label frequencies show moderate class imbalance. Neutrophils (19.48% of all samples) and Eosinophils (18.24%) are the largest classes, while Lymphocytes (7.10%) and Basophils (7.13%) are the smallest. Figure 1 shows this imbalance directly. Because accuracy can hide minority-class degradation, later evaluation emphasizes macro-averaged F1-score and minimum per-class F1-score.

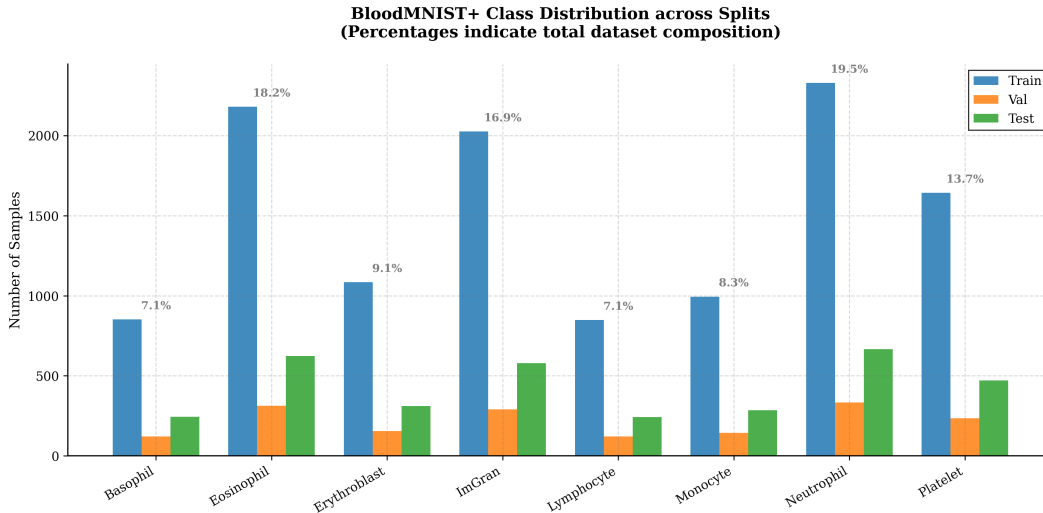


Figure 1: BloodMNIST class-distribution profile, showing moderate imbalance across the eight blood-cell classes.

The image preprocessing was matched to the ImageNet pretraining setup. Each image was converted to three RGB channels, resized to 224×224 , and standardized using the ImageNet normalization vectors

$$\mu_{\text{ImageNet}} = [0.485, 0.456, 0.406], \quad \sigma_{\text{ImageNet}} = [0.229, 0.224, 0.225]. \quad (2.6)$$

Here, μ_{ImageNet} is the three-channel mean vector and σ_{ImageNet} is the matching standard deviation vector for RGB inputs. Figure 2 shows the RGB intensity distribution before normalization. This step centers and scales the image covariates channel by channel, keeps the input scale close to the ImageNet pretraining distribution, and makes baseline and compressed models easier to compare fairly.

Table 1: Class distribution of the BloodMNIST dataset across the train, validation, and test splits.

Cell Class	Train	Val	Test	Total	Share
Basophil	852	122	244	1,218	7.13%
Eosinophil	2,181	312	624	3,117	18.24%
Erythroblast	1,085	155	311	1,551	9.07%
Immature Granulocytes	2,026	290	579	2,895	16.94%
Lymphocyte	849	122	243	1,214	7.10%
Monocyte	993	143	284	1,420	8.31%
Neutrophil	2,330	333	666	3,329	19.48%
Platelet	1,643	235	470	2,348	13.74%
Total	11,959	1,712	3,421	17,092	100.00%

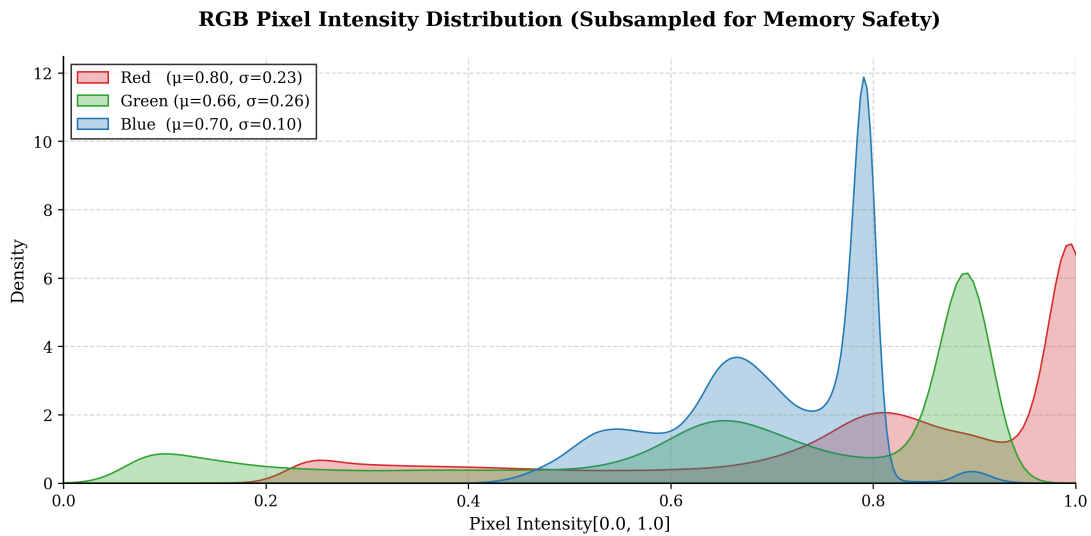


Figure 2: RGB pixel-intensity distributions before ImageNet normalization. The channel distributions motivate standardized preprocessing before VGG19 fine-tuning.

Beyond label frequencies, the exploratory analysis also shows the visual structure of the classification task. Figure 3 summarizes representative BloodMNIST examples by visual complexity.

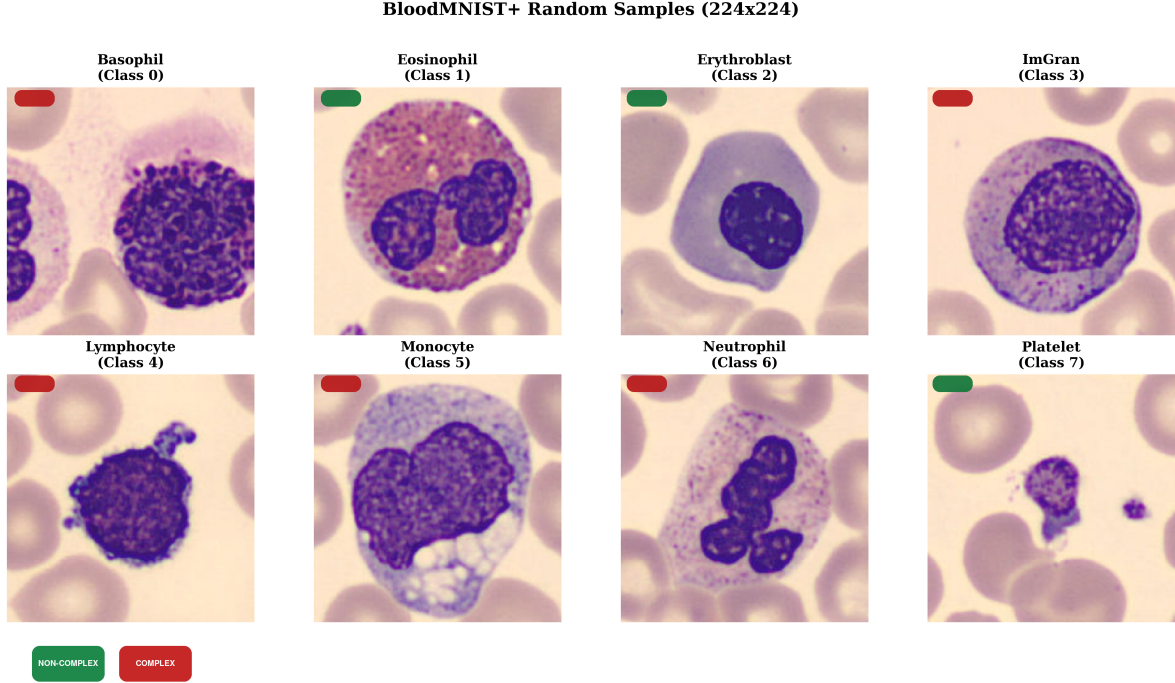


Figure 3: Representative BloodMNIST cell samples labelled by visual complexity. Green labels indicate visually clearer classes, while red labels indicate classes expected to be more sensitive to compression.

In Figure 3, green marks classes with clearer morphology, while red marks classes that require finer discrimination because of overlap in granularity, nuclear structure, staining, or cell size. These criteria are consistent with standard peripheral-blood-smear interpretation, where cell identification depends on size, staining, granularity, and nuclear morphology (Lynch, 1990). This makes the dataset a useful test for statistical compression: clearer classes should be preserved first, while information loss should appear earlier in visually ambiguous classes.

2.3 Second-Order Pruning as a Theoretical Baseline

The statistical reason for removing a weight is sensitivity. If deleting a weight changes the loss very little, that weight is a good pruning candidate. This idea can be written using a second-order Taylor expansion of the loss function.

If the network is near a local minimum, the first-order gradient is close to zero ($\nabla L \approx 0$). Then the change in loss ΔL caused by a weight change Δw can be approximated using the Hessian matrix H :

$$\Delta L \approx \frac{1}{2} \Delta w^T H \Delta w \quad (2.7)$$

where ΔL denotes the change in loss, Δw is the perturbation applied to the weight vector, and H

is the Hessian matrix of second-order loss derivatives with respect to the weights.

In the Optimal Brain Surgeon (OBS) framework, the estimated loss increase from pruning a weight w_q , while allowing the other weights to adjust, is based on the inverse Hessian:

$$L_q = \frac{w_q^2}{2[H^{-1}]_{qq}} \quad (2.8)$$

where L_q is the estimated loss increase caused by pruning weight w_q , and $[H^{-1}]_{qq}$ is the q th diagonal element of the inverse Hessian, representing curvature-adjusted sensitivity for that weight.

OBS gives a useful theoretical benchmark, but computing the inverse Hessian for a modern model such as 140-million-parameter VGG19 is not practical. This study therefore uses scalable statistical approximations: L_1 regularization, L_0 stochastic relaxation, and singular value decomposition. These methods aim for the same broad goal—removing low-value model capacity—without the cost of full second-order pruning. This framing is consistent with sparse-learning work that treats compression as a tractable approximation to more expensive Bayesian or variational selection procedures (Bai, Song, & Cheng, 2020).

2.4 Scalable Statistical Compression Methods

The following subsections define the three compression methods. Exact optimization schedules and the rationale for selecting learning rates, penalties, thresholds, and guardrails are reported in Appendix B.

2.4.1 Sparse Regularization via L_1 Norm (Lasso)

To create unstructured sparsity during training, we use the L_1 norm penalty, or Lasso. Unlike L_2 weight decay, which shrinks weights continuously, the L_1 penalty can drive small weights toward zero and follows the Lasso idea of shrinkage and variable selection (Tibshirani, 1996).

The regularization term $\lambda\|w\|_1$ adds a constant pull toward zero. The update rule is:

$$w_j^{\text{new}} \leftarrow w_j - \eta \nabla L_j - \eta \lambda \text{sign}(w_j) \quad (2.9)$$

where η is the learning rate, ∇L_j is the derivative of the data-fit loss with respect to weight w_j , and λ controls the sparsity penalty. If a weight contributes little to reducing the loss, the penalty can dominate and move it toward zero. This helps identify influential weights and supports later magnitude pruning.

2.4.2 Differentiable L_0 Regularization and Gaussian Gates

While L_1 is a convex approximation, the L_0 norm ($\|w\|_0 = \sum \mathbb{I}[w_j \neq 0]$) is a more direct compression target. It counts how many parameters are non-zero. Unlike L_1 , it does not penalize large useful weights more heavily just because they are large.

The problem is that the discrete L_0 norm is not differentiable. Louizos et al. (2018) addressed this with a continuous relaxation based on the Hard Concrete distribution. In our pretrained VGG19 setting, however, this created a practical problem: the gate parameters started too far from the open/closed decision boundary. Closing gates required strong regularization, which risked destabilizing the pretrained convolutional weights.

To address this, we use a Gaussian continuous relaxation. Each stochastic gate is controlled by μ and σ , and the gate value during training is:

$$z = \text{clamp}(\mu + \sigma\epsilon + 0.5, 0, 1), \quad \epsilon \sim \mathcal{N}(0, 1) \quad (2.10)$$

where z is the stochastic gate value, ϵ is standard-normal noise, μ controls the gate location, and σ controls the noise scale. The operator $\text{clamp}(a, 0, 1)$ bounds a below by 0 and above by 1; it is the same operation often called $\text{clip}(a, 0, 1)$ in numerical computing. Although classical additive-error notation often writes the noise term directly as $\mathcal{N}(0, \sigma^2)$, here ϵ is standardized as $\mathcal{N}(0, 1)$ and then multiplied by σ ; therefore, $\sigma\epsilon \sim \mathcal{N}(0, \sigma^2)$. This reparameterized form is equivalent to drawing the pre-clamped gate from $\mathcal{N}(\mu + 0.5, \sigma^2)$, but it isolates the randomness in ϵ while keeping the dependence on μ and σ differentiable. Initializing $\mu = 0.5$ keeps gates open at the start while placing them near the decision boundary. The expected L_0 complexity is then written using the Gaussian cumulative distribution function Φ :

$$\mathbb{E}[\|z\|_0] \approx \sum_i \Phi\left(\frac{\mu_i + 0.5}{\sigma}\right) \quad (2.11)$$

where the sum is over gates i . Minimizing this penalty moves unnecessary gates toward negative μ values. Those channels can then be closed and physically removed during reconstruction.

2.4.3 Dimensionality Reduction via Singular Value Decomposition (SVD)

While L_1 and L_0 focus on sparsity, truncated singular value decomposition (SVD) targets low-rank structure in dense fully connected (FC) layers.

Any weight matrix $W \in \mathbb{R}^{m \times n}$ can be decomposed into $W = U\Sigma V^T$. By retaining only the top k singular values, we approximate the original matrix as:

$$W \approx U_k \Sigma_k V_k^T \quad (2.12)$$

where W is the original dense weight matrix, U_k and V_k contain the retained singular vectors, Σ_k contains the retained top- k singular values, and $k < \min(m, n)$ is the selected rank. The Eckart–Young–Mirsky theorem shows that this is the best rank- k approximation under the Frobenius norm (Eckart & Young, 1936; Mirsky, 1960). SVD uses this structure to replace large matrices with

smaller factorized linear layers while preserving the main directions of variation.

Figure 5 shows why SVD is suitable for the dense classifier. The singular values fall quickly, so most matrix energy is concentrated in a small number of directions. The long tail contributes less to the representation.

The singular-value diagnostic also gives a practical interpretation of the selected rank cap. The largest singular values were clearly separated from the tail: FC0 began with a leading value of 5.13 followed by values around 3.80–3.40, while FC3 began with a leading value of 6.62 followed by values around 3.42–3.17. This suggests that both layers contain dominant spectral directions, but the remaining spectrum is still broad rather than negligible. During computation, the first two large FC layers produced warnings because a rank limit of 1024 did not reach the 95% cumulative-energy target. For FC0, the first 1024 singular values explained 69.7% of the energy; for FC3, they explained 81.1%. Thus, the rank-1024 setting should be interpreted as a computationally constrained compression choice rather than as a full 95%-energy reconstruction for these layers. Even so, replacing a 4096-rank matrix by rank 1024 makes each layer 75.0% rank-reducible in rank count, showing substantial structural compression.

Table 2: SVD rank-cap diagnostic for fully connected layers. The 1024-rank cap reduces rank count substantially but does not reach the 95% cumulative-energy target for the two largest FC matrices.

Layer	Matrix Size	Rank Cap	Energy at Cap	Rank Reduction
FC0	25088 $\rightarrow$ 4096	1024	69.7%	75.0%
FC3	4096 $\rightarrow$ 4096	1024	81.1%	75.0%

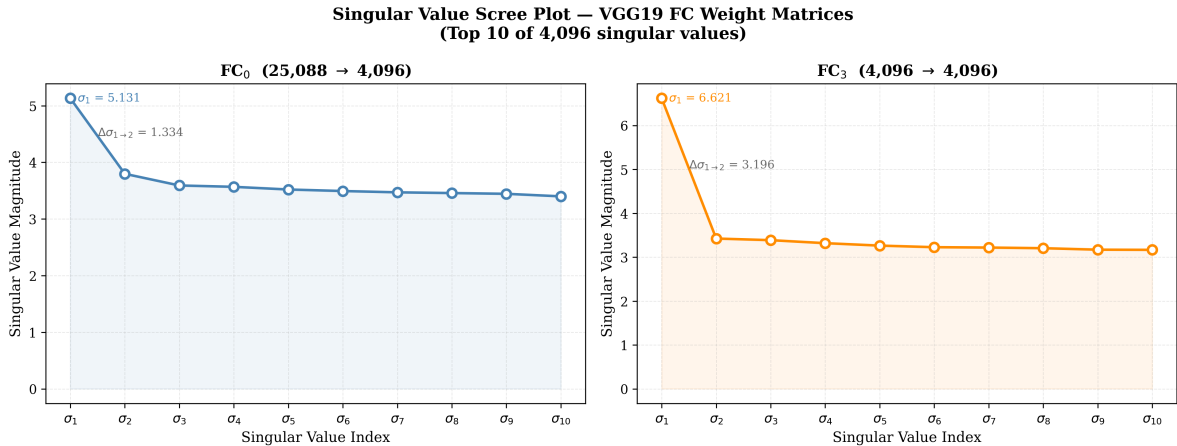


Figure 4: SVD scree plot for fully connected layers. The cumulative-energy curves show how much spectral information is retained as the selected rank increases, supporting the interpretation of the 1024-rank cap.

2.5 Generalization and Model Complexity Motivation

These compression methods are motivated by statistical learning theory, which links model complexity to test performance. The following ideas are used as motivation, not as formal finite-sample

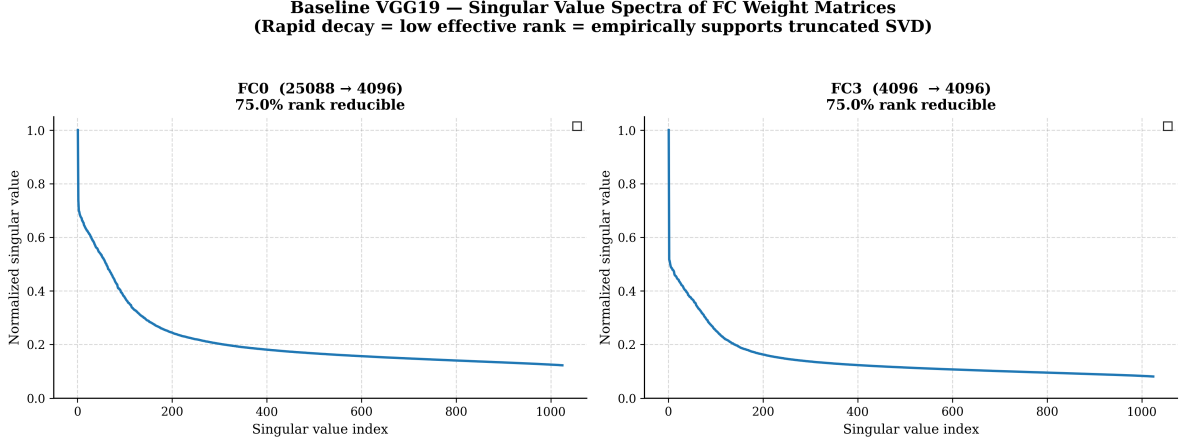


Figure 5: SVD singular-value spectra for fully connected layers. Rapid spectral decay indicates that the dense classifier contains low-rank redundancy suitable for factorization.

guarantees for VGG19.

Minimum Description Length (MDL) principle The MDL principle favors models that describe the data well using a short model description (Rissanen, 1978). In this study, L_0 pruning reduces the number of active components, and SVD reduces matrix rank. If predictive fit is preserved, the compressed model has a better complexity–fit trade-off.

VC-dimension reduction Pruning reduces the number of active parameters and can reduce the effective complexity of the hypothesis class, following the statistical-learning idea that generalization depends partly on hypothesis-class capacity (Vapnik & Chervonenkis, 1971; Bartlett et al., 2021). A standard VC-style bound relates expected risk $R(f)$ to empirical risk $R_{\text{emp}}(f)$ and a complexity term:

$$R(f) \leq R_{\text{emp}}(f) + O\left(\sqrt{\frac{d_{VC} \log n}{n}}\right) \quad (2.13)$$

where $R(f)$ is expected risk, $R_{\text{emp}}(f)$ is training risk, d_{VC} is the Vapnik–Chervonenkis dimension, and n is the number of training samples. The bound is not used numerically; it motivates the idea that lower effective complexity can support better generalization when data fit is preserved.

The lottery ticket hypothesis This hypothesis gives another reason to expect useful sparse subnetworks: a dense network may contain a smaller subnetwork that can match the baseline (Frankle & Carbin, 2019). In this study, L_1 Lasso regularization and L_0 Gaussian gates are treated as statistical search tools for finding such subnetworks.

Chapter 3

Baseline Model Analysis

3 Baseline Model Analysis

3.1 Introduction

This chapter establishes the empirical baseline for the study by evaluating the uncompressed VGG19 model on BloodMNIST. The baseline defines the reference level of predictive performance, class-wise stability, memory use, latency, and statistical complexity before compression. Because BloodMNIST has visual overlap and class imbalance, the model is assessed with macro-level and class-level metrics rather than overall accuracy alone.

3.2 Architectural Analysis of VGG19

VGG19 can be interpreted as a non-linear, non-parametric statistical estimator. Architecturally, it consists of sixteen convolutional layers and three fully connected layers (Simonyan & Zisserman, 2015). Figure 6 shows the canonical ImageNet VGG architecture used as the starting point; in this study, the final classification layer is adapted from the original 1000 ImageNet classes to the eight BloodMNIST classes.

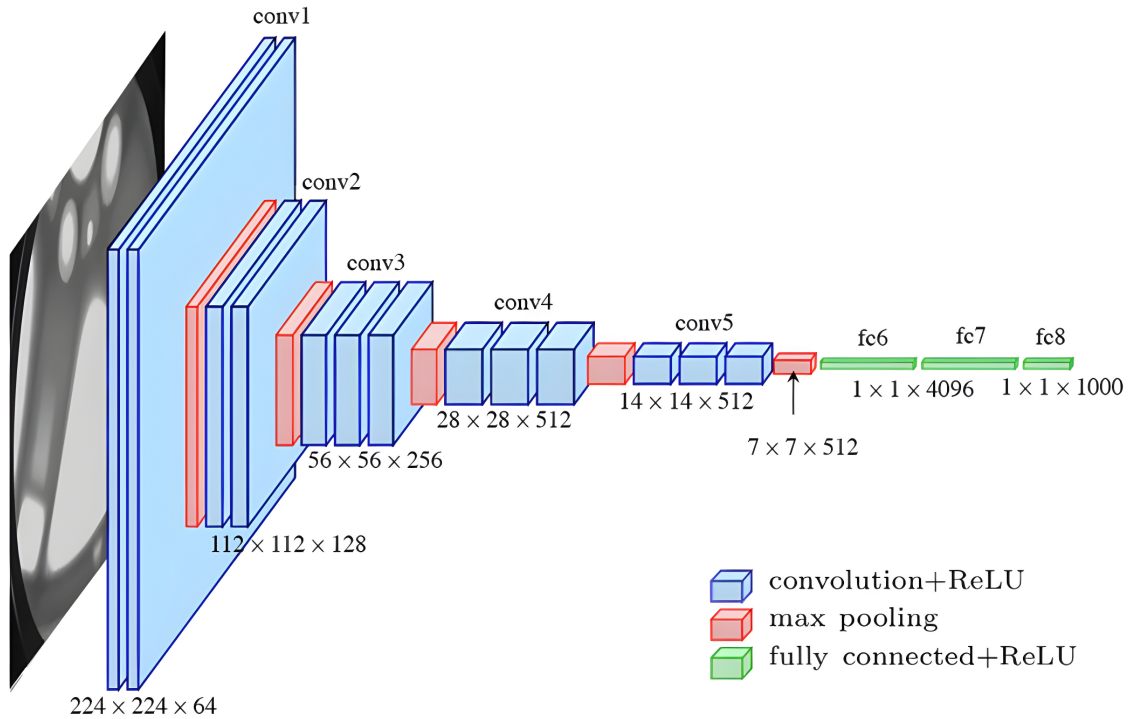


Figure 6: VGG19 architecture schematic. The diagram shows the canonical ImageNet-style VGG feature extractor and classifier layout; the present study uses this architecture as the baseline but replaces the final output layer with an eight-class BloodMNIST classifier, following the VGG design of Simonyan and Zisserman (2015).

The model contains 139,603,016 learnable parameters, but this capacity is distributed unevenly.

The convolutional base contains approximately 2.01×10^7 parameters, while the fully connected classification head contains more than 1.196×10^8 parameters, or about 85.6% of the total. This imbalance is important for compression: although the convolutional layers carry much of the spatial feature extraction, most of the parameter burden lies in the dense classifier. The fully connected head is therefore a natural target for low-rank compression and redundancy analysis.

For BloodMNIST, the training set contains $N_{\text{train}} = 11,959$ samples, giving a parameter-to-sample ratio above 11,600:1. This makes VGG19 a useful baseline for testing whether compression can remove excess complexity while preserving predictive signal.

Figure 7 provides empirical evidence of activation-space redundancy in the baseline model. The PCA curves saturate quickly, indicating that much of the activation variance is captured by fewer directions than the nominal layer widths suggest. This supports the interpretation that the model has a lower effective dimension than its raw parameter count implies.

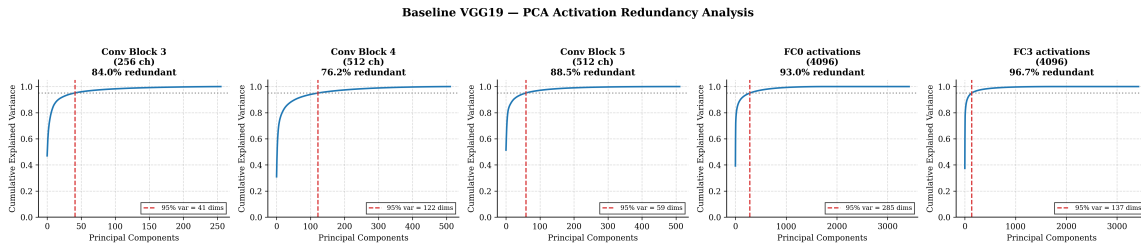


Figure 7: Baseline activation-space PCA, showing rapid variance saturation and supporting the claim that VGG19 has lower effective activation dimension than its raw width suggests.

3.3 Fine-Tuning Methodology

To obtain a stable and reproducible baseline, VGG19 was initialized with ImageNet-1K pretrained weights. Transfer learning provides low-level visual filters before BloodMNIST training begins, reducing the amount of structure that must be estimated from the BloodMNIST sample alone.

Images were resized to 224×224 pixels and standardized using ImageNet mean and standard deviation values. The network was optimized with stochastic gradient descent (SGD), Nesterov momentum of 0.9, and an initial learning rate of $\eta = 1 \times 10^{-3}$. A step-decay schedule used decay factor $\gamma = 0.1$ at fixed epoch intervals. The objective function was categorical cross-entropy, equivalent to the negative log-likelihood for a multinomial classification model. Training ran for 15 epochs, and the selected checkpoint was the epoch with the highest validation macro-F1 on the validation set ($N_{\text{val}} = 1,712$). The full baseline optimization settings and epoch-level training log are reported in Appendix B.

Figure 8 summarizes the baseline optimization trajectory over 15 epochs, showing convergence, stable validation performance, and acceptable minimum per-class F1.

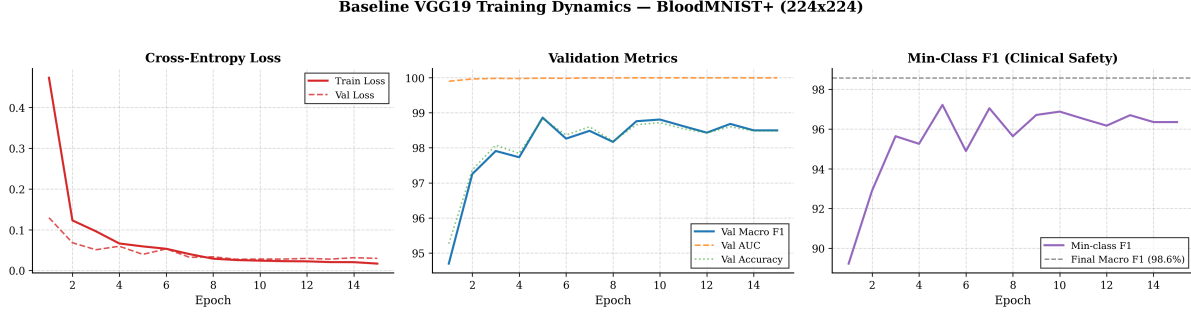


Figure 8: Baseline VGG19 training dynamics. Validation metrics stabilize quickly, with strong macro-F1 and minimum-class F1 before later compression experiments.

3.4 Empirical Performance and Class-Level Behavior

The trained baseline was evaluated on the held-out blind test set ($N_{\text{test}} = 3,421$). Its performance provides the reference point for judging whether later compression methods preserve diagnostic fidelity.

3.4.1 Macro-Level Statistical Metrics

At the aggregate level, the baseline model performed strongly. It achieved 98.48% top-1 accuracy, 98.57% macro-averaged F1-score, and 99.94% macro ROC-AUC under a one-vs-rest evaluation. The Matthews Correlation Coefficient (MCC) and Cohen’s Kappa (κ) were both 98.23%, and the mean test loss was 0.0488. The close agreement between accuracy and macro-F1 indicates that performance is not driven only by the majority classes, while the high Kappa value indicates strong agreement beyond chance.

Figure 9 shows that baseline errors are rare and concentrated mainly in morphologically complex classes, especially Immature Granulocytes.

Figure 10 summarizes class separability using one-vs-rest ROC curves, showing that the uncompressed VGG19 model provides a strong discriminative reference point before compression.

3.4.2 Per-Class Evaluation

The weakest class-level result was for Immature Granulocytes, with an F1-score of 96.28%. Because this class has high within-class variation, it acts as a useful stress class for later compression experiments.

Figure 11 summarizes the baseline per-class F1 profile, with the small gap from the macro average mainly associated with morphologically ambiguous classes.

These baseline results should be interpreted within the limits of the independent and identically

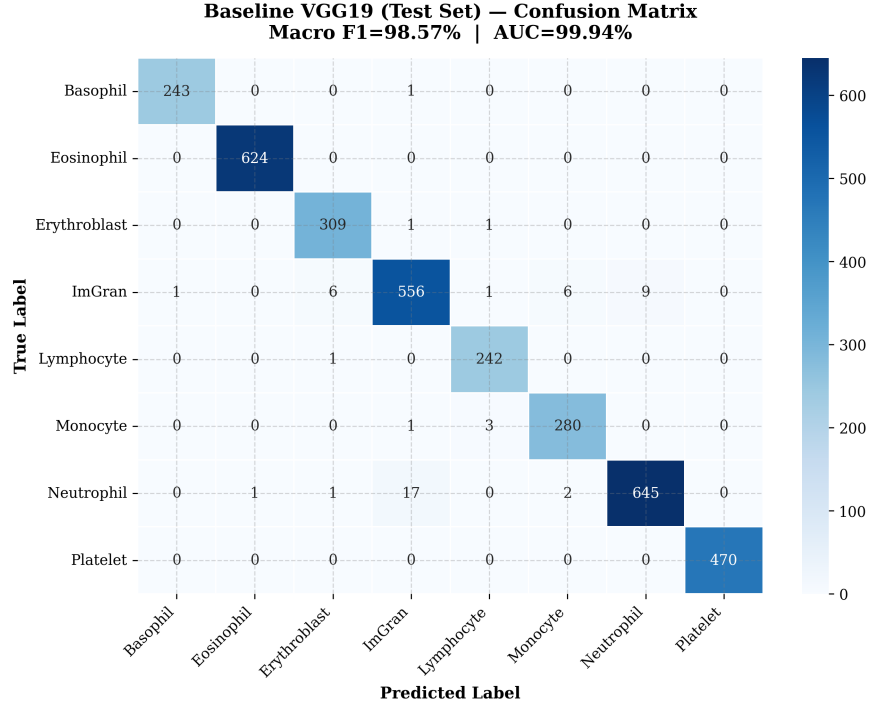


Figure 9: Baseline confusion matrix. The strong diagonal indicates high diagnostic fidelity, while the remaining errors identify morphologically difficult classes for later comparison.

Table 3: Class-wise diagnostic performance of the baseline VGG19 architecture.

Cell Class	Support	Precision	Recall	F1	Acc
Basophil	244	99.59%	99.59%	99.59%	99.97%
Eosinophil	624	99.92%	99.84%	100.00%	99.96%
Erythroblast	311	98.41%	97.48%	99.36%	99.74%
Immature Granulocytes	579	96.28%	96.53%	96.03%	99.30%
Lymphocyte	243	98.78%	97.98%	99.59%	99.84%
Monocyte	284	97.90%	97.22%	98.59%	99.74%
Neutrophil	666	97.73%	98.62%	96.85%	99.67%
Platelet	470	100.00%	100.00%	100.00%	100.00%

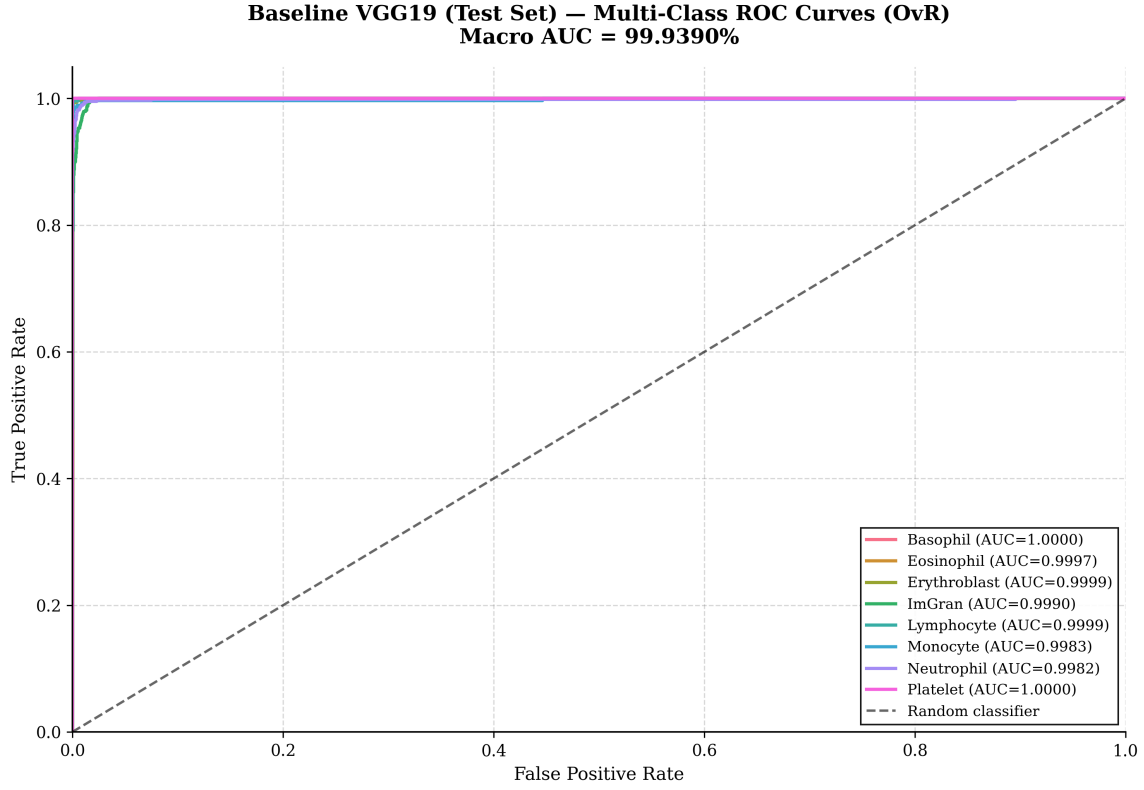


Figure 10: Baseline multi-class ROC curves. Near-saturated one-vs-rest curves establish high class separability before compression.

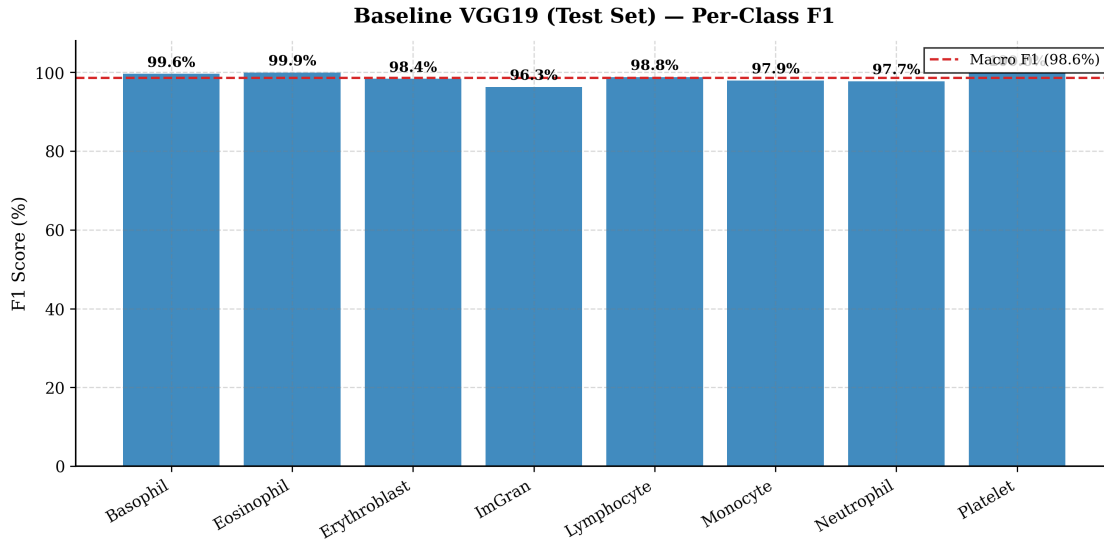


Figure 11: Baseline class-wise F1-score distribution. The small spread around the macro average shows that the uncompressed model is not relying only on majority classes.

distributed (IID) assumption used in standard statistical learning. The empirical test metrics estimate performance under the assumption that the training, validation, and test observations are sampled from the same data-generating distribution,

$$(X_i, Y_i) \stackrel{\text{iid}}{\sim} P_{\text{BloodMNIST}}, \quad i = 1, \dots, N.$$

This assumption is reasonable for the curated BloodMNIST split, but it may not hold for images collected under different staining conditions, devices, preparation protocols, or population settings. In those cases, the deployment distribution may differ from the study distribution, so that $P_{\text{deploy}}(X, Y) \neq P_{\text{BloodMNIST}}(X, Y)$. Such samples are out-of-distribution (OOD), and reliability cannot be assumed from in-distribution test performance alone. This limitation is consistent with statistical views of deep learning and recent work showing that strong in-distribution performance does not automatically imply robust OOD generalization (Goodfellow, Bengio, & Courville, 2016; Bartlett et al., 2021; Geirhos et al., 2020; Aaronson, Lee, & Li, 2026).

3.5 Computational Complexity Assessment

Although the baseline model predicts accurately, its deployment cost is substantial. This matters because the goal of compression is not only to preserve classification performance, but also to reduce storage, memory use, latency, and statistical complexity.

With 32-bit floating-point weights, the uncompressed VGG19 model requires 532.5 MB of storage. During forward evaluation, its peak memory footprint is 3,249.4 MB, and its inference latency is 195.7 ± 14.8 milliseconds per batch. These values establish the computational baseline against which compressed models are compared.

To measure the trade-off between fit and complexity, the Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC) were also computed. Using the mean test loss of 0.0488, the empirical log-likelihood (L) was estimated as -166.85 . With approximately 1.396×10^8 parameters, this gives an AIC of 279,206,365.70 and a BIC of 1,136,046,147.59. These values are dominated by the parameter count, which makes them useful indicators of the complexity penalty that compression aims to reduce.

3.6 Baseline Summary

The baseline VGG19 model achieves strong predictive performance on BloodMNIST, with 98.48% accuracy, 98.57% macro-F1, and stable class-level behavior. However, this performance comes with high storage cost, high memory use, measurable inference latency, and very large AIC and BIC values driven by the parameter count. These findings support the central premise of the study: the uncompressed model is accurate, but it likely contains statistical redundancy that compression may remove while preserving most of the predictive signal.

Chapter 4

Compression Results and Structural Analysis

4 Compression Results and Structural Analysis

This chapter evaluates the three compression methods as a sequence of statistical model-reduction steps. The first method, L_1 Lasso regularization, tests whether many individual VGG19 weights can be shrunk toward zero with little loss in prediction. The second method, Singular Value Decomposition (SVD), tests whether the dense classification layers can be represented by lower-rank matrices. The final method, L_0 Gaussian gating, tests whether whole convolutional channels can be removed and rebuilt into a physically smaller network. Together, these methods compare unstructured sparsity, low-rank approximation, and structured channel pruning.

4.1 L1 Lasso Regularization

4.1.1 L1 Rationale and Training Configuration

The first experiment used L_1 Lasso regularization as a direct test of weight-level redundancy. The mathematical form of the L_1 penalty is introduced in Chapter 2; here, the method is treated as a statistical shrinkage procedure that pushes weakly useful weights toward zero during training. This produces unstructured sparsity, where individual weights are selected or suppressed independently.

The regularization coefficient (λ) was increased linearly from 1×10^{-5} to 3×10^{-3} across training. Layer-wise multipliers were based on the earlier PCA findings: fully connected layers received a stronger $5.0\times$ penalty, while early convolutional layers received a smaller $0.5\times$ penalty to protect low-level image features. The model was optimized using stochastic gradient descent with warm restarts (SGDR), a base learning rate of 5×10^{-4} , and restart interval $T_0 = 10$ epochs. Appendix B gives the full L_1 schedule, IMP refinement settings, and hyperparameter rationale.

4.1.2 Training Dynamics and Sparsity Transition

The training path showed the trade-off between predictive fit and sparsity. During the early epochs, the model tolerated the increasing λ penalty and accumulated soft sparsity quickly. A sharp drop occurred at epoch 22: validation F1 fell to 72.23%, which triggered early stopping, and the minimum per-class F1 fell to 23.2%. Statistically, this indicates that the regularization term had moved beyond harmless shrinkage and had begun to remove weights needed for class separation.

The full epoch-by-epoch trajectory, including the evolving λ penalty, validation loss, minimum per-class F1-score, and soft sparsity, is reported in Appendix B (Table 22). This log shows why epoch 20 was retained as the final dense-sparse checkpoint.

At epoch 20, the model reached 92.5% soft sparsity while maintaining a validation F1-score of 98.19%. The weight distribution also supported the sparsity interpretation: 94.7% of parameters fell below the hard threshold of 1×10^{-3} , while a smaller tail of larger weights remained available to carry the discriminative signal.

L1 Lasso Training Dynamics — VGG19 on BloodMNIST+

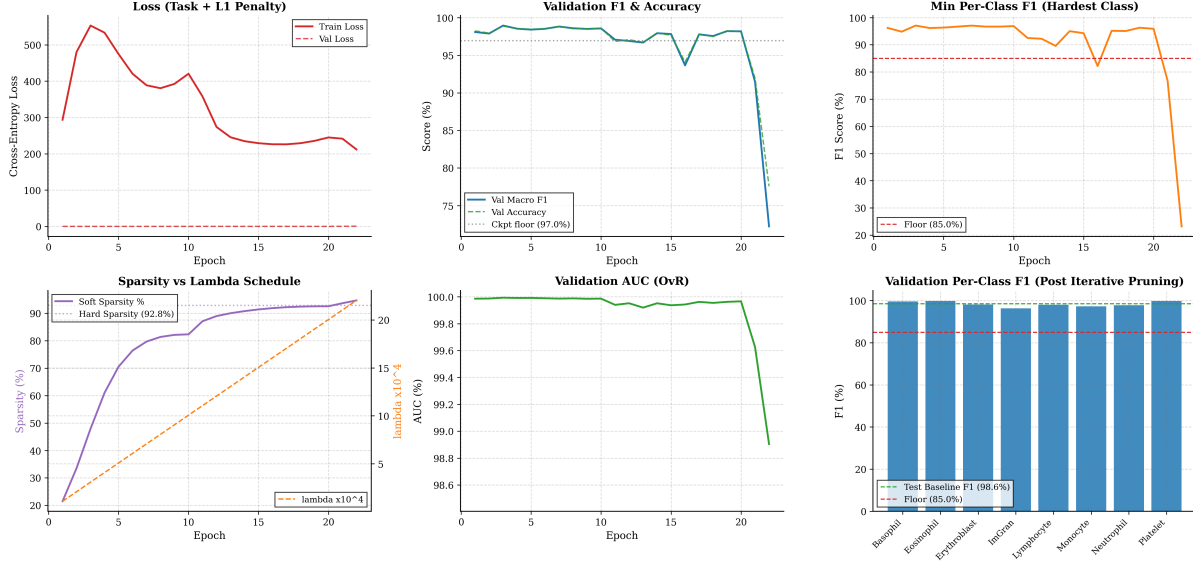


Figure 12: L1 Lasso training dynamics. The curves show the trade-off between increasing sparsity and validation stability, including the late phase collapse that motivated early stopping.

L1 Lasso — Weight Magnitude Distribution (Post-Training, Pre-Surgery)

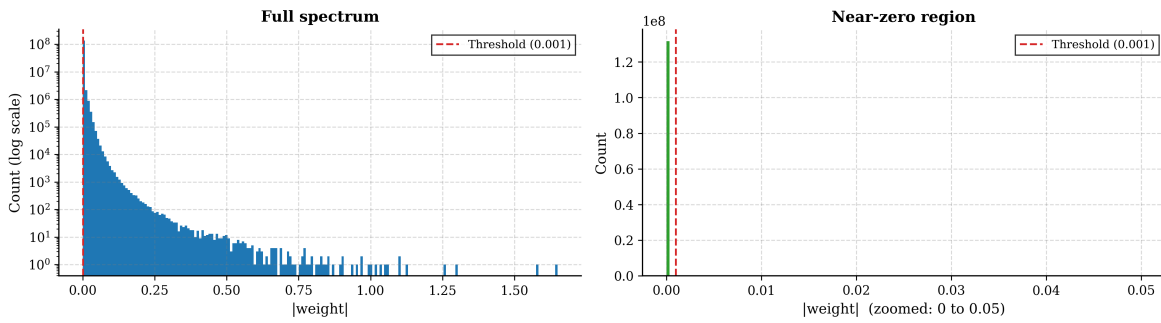


Figure 13: Post-L1 weight magnitude distribution. The large near-zero mass supports converting soft shrinkage into hard sparsity through magnitude pruning.

4.1.3 Iterative Magnitude Pruning (IMP) and Structural Refinement

Soft sparsity does not automatically remove weights from the stored model. Therefore, the epoch 20 checkpoint was converted into hard sparsity using iterative magnitude pruning (IMP), where low-magnitude weights were set to zero and masked during refinement.

The first IMP cycle converted the 92.5% soft sparsity into hard zeros. After a short disruption, five epochs of masked refinement recovered validation F1 to 98.52%. The second cycle kept the same threshold and mainly stabilized the mask. In the third cycle, the threshold multiplier was increased to $1.5\times$, raising hard sparsity to 92.8% and reducing the active parameter count to 10.00 million. After three final refinement epochs, the model remained stable, suggesting that the remaining 7.2% of parameters were sufficient for this benchmark.

The full masked-refinement history across all three IMP cycles is provided in Appendix B (Table 25).

Table 4: Iterative magnitude pruning and refinement results for the L1 Lasso model.

Cycle	Thresh. Mult.	Active Params.	Hard Sparsity	Post-Surg. Val. F1	Best Refined Val. F1
Cycle 1	$1.0\times$	10.41M	92.5%	98.25%	98.52%
Cycle 2	$1.0\times$	10.40M	92.5%	98.52%	98.26%
Cycle 3	$1.5\times$	10.00M	92.8%	98.14%	98.32%

4.1.4 Empirical Evaluation and Class-Level Behavior

On the blind test set ($N = 3,421$), the final L_1 -regularized model preserved most of the baseline performance. It achieved 98.30% top-1 accuracy and 98.29% macro-F1. Relative to the uncompressed baseline macro-F1 of 98.57%, this is a decrease of only 0.28 percentage points while removing 92.8% of active weights.

The macro ROC-AUC remained 99.94%, matching the baseline. Class-wise performance was also stable. The most morphologically variable class, Immature Granulocytes, retained an F1-score of 96.36%, remaining well above the 90% interpretive guardrail used in this study.

Table 5: Class-wise diagnostic comparison between the baseline model and the L1 Lasso model.

Cell Class	Baseline F1	L1 Lasso F1	Δ F1	Baseline Recall	L1 Lasso Recall
Basophil	99.59%	98.77%	-0.82%	99.17%	98.36%
Eosinophil	99.92%	99.84%	-0.08%	99.68%	100.00%
Erythroblast	98.41%	98.07%	-0.34%	98.07%	98.07%
Immature Granulocytes	96.28%	96.36%	+0.08%	96.70%	96.03%
Lymphocyte	98.78%	98.56%	-0.22%	98.36%	98.77%
Monocyte	97.90%	97.04%	-0.86%	95.88%	98.24%
Neutrophil	97.73%	97.89%	+0.16%	98.33%	97.45%
Platelet	100.00%	99.79%	-0.21%	99.58%	100.00%

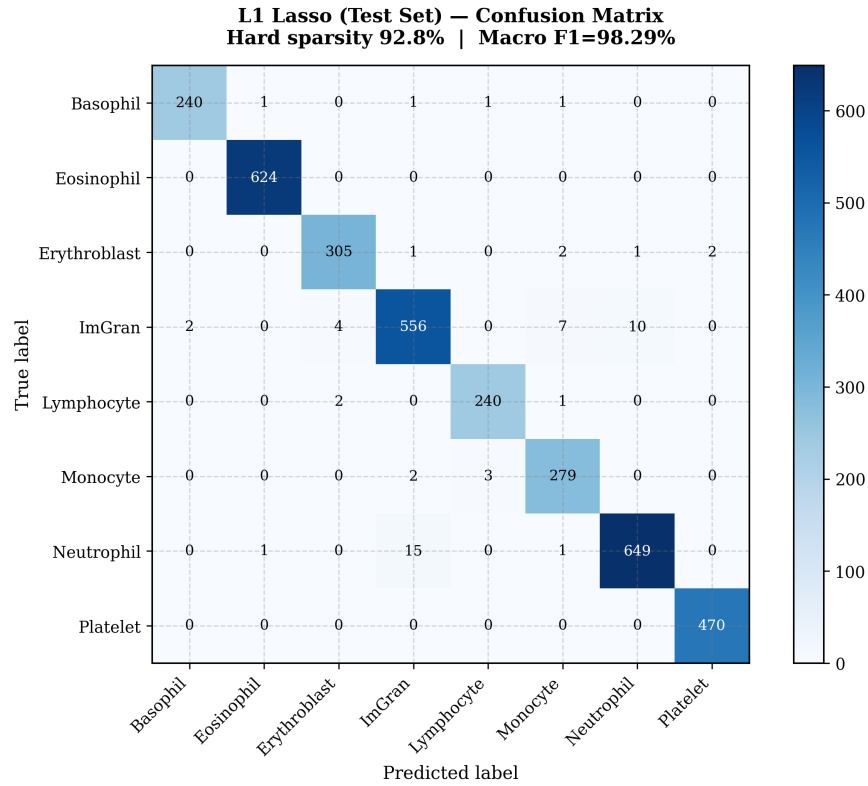


Figure 14: L1 Lasso confusion matrix. The diagonal remains close to the baseline, indicating that 92.8% unstructured sparsity preserved most class boundaries.

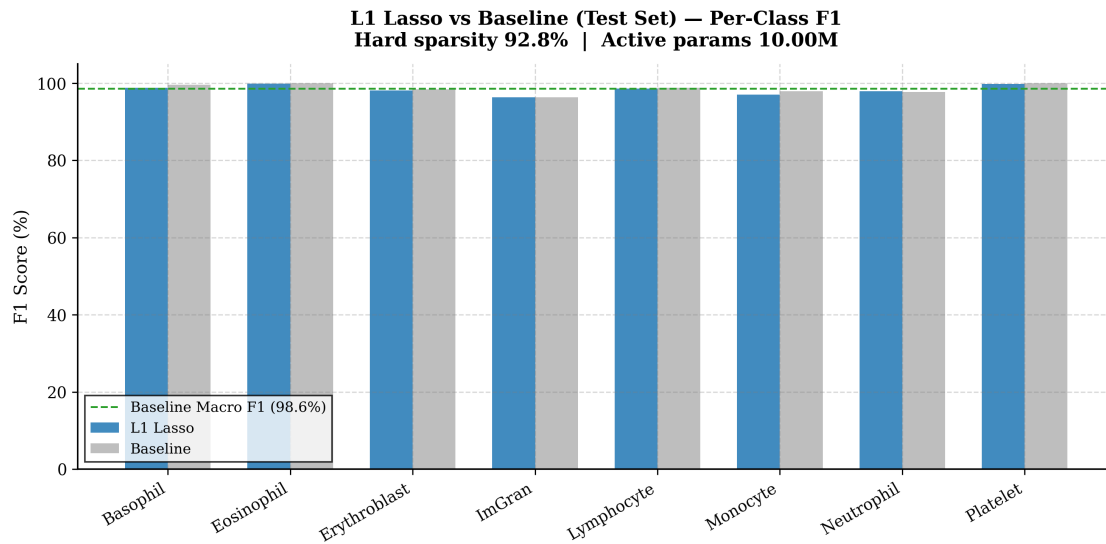


Figure 15: Baseline and L1 per-class F1-score comparison. Small class-wise changes show that L1 sparsity mainly removed redundant weights rather than collapsing a specific diagnostic class.

4.1.5 Statistical Efficiency vs. Hardware Limitations

By reducing the active parameter count from approximately 139.6 million to 10.0 million, L_1 regularization produced a much more parsimonious statistical model. The Akaike Information Criterion (AIC) fell from 2.79×10^8 for the baseline to 2.00×10^7 , and the Bayesian Information Criterion (BIC) dropped to 8.13×10^7 . These criteria support the conclusion that many baseline parameters were not needed for the BloodMNIST decision rule.

However, the deployment measurements showed a limitation of unstructured sparsity. Despite eliminating more than 129 million active weights, inference latency increased from 195.7 to 207.0 milliseconds per batch, and GPU memory usage increased to 4,854.0 MB. The reason is structural: the tensor shapes did not change, so standard dense hardware still evaluated the same matrix dimensions unless specialized sparse kernels were used.

Thus, L_1 Lasso provided strong statistical evidence of over-parameterization with minimal accuracy loss, but it did not provide hardware acceleration. The next method therefore asks whether redundancy can be removed by changing matrix rank rather than only zeroing individual weights. This motivates SVD, where dense classification layers are replaced by lower-rank factors.

4.2 Dimensionality Reduction via Singular Value Decomposition (SVD)

4.2.1 Theoretical Rationale and Experimental Configuration

The L_1 experiment showed that many individual weights could be removed with little loss in test performance, but its zeroed weights remained scattered inside dense tensors. SVD was used next because it changes the matrix representation itself, replacing large dense matrices with lower-rank factors.

The earlier PCA audit showed that most variance redundancy was concentrated in the fully connected classification head. The FC0 layer ($25,088 \times 4,096$) and FC3 layer ($4,096 \times 4,096$) together accounted for about 89% of the model parameters and showed more than 93% activation redundancy.

Truncated SVD provides a statistically controlled way to reduce these layers. Using the Eckart–Young–Mirsky theorem (Eckart & Young, 1936; Mirsky, 1960), each selected weight matrix was decomposed into orthogonal directions ranked by singular-value energy. Each dense layer was then replaced by two smaller linear layers without an intervening non-linearity, preserving a low-rank approximation of the original matrix.

4.2.2 Energy Threshold Sweep and Rank Selection

To choose the rank k , an energy-threshold sweep was run on the validation set before fine-tuning. The goal was to find a threshold that reduced parameters substantially while keeping macro-F1 above the 85.0% clinical guardrail.

Table 6: SVD energy threshold sweep before fine-tuning on the validation set.

Threshold	Rank FC0	Rank FC3	Active Params	Reduction	Val F1	Val AUC	Min F1
0.50	309	169	30.47M	78.2%	98.82%	99.98%	96.7%
0.40	150	92	25.20M	82.0%	98.72%	99.98%	96.4%
0.30	79	55	22.82M	83.7%	98.46%	99.98%	95.9%
0.20	44	31	21.60M	84.5%	95.32%	99.89%	88.8%
0.10	19	12	20.72M	85.2%	43.70%	85.77%	0.0%

The sweep showed a clear stability boundary. At the 0.20 threshold, validation macro-F1 remained 95.32%, but at 0.10 it collapsed to 43.70%. The 0.20 threshold was therefore selected. At this setting, FC0 was reduced from rank 4,096 to 44, and FC3 was reduced from rank 4,096 to 31. This removed 84.5% of the model parameters before fine-tuning while preserving enough singular directions for classification.

4.2.3 Fine-Tuning Dynamics

After truncation, the factorized model was fine-tuned for 10 epochs. The convolutional base was frozen so that only the new low-rank classification head adapted to the existing feature representation.

The optimizer was Adam with learning rate 2×10^{-4} . Appendix B reports the SVD factorization parameters, fine-tuning settings, and threshold-selection rationale.

Fine-tuning was stable. The best validation checkpoint occurred at epoch 8, with macro-F1 of 98.86% and minimum per-class F1 of 96.9%. The complete epoch-level log is reported in Appendix B (Table 27). The fast recovery suggests that many discarded singular directions carried little class-relevant signal for this dataset.

4.2.4 Empirical Evaluation and Class-Level Behavior

On the blind test set ($N = 3,421$), the SVD-compressed model achieved 98.71% top-1 accuracy and 98.77% macro-F1. This was 0.20 percentage points above the baseline macro-F1, despite removing 84.5% of the model parameters.

This result is consistent with the regularization effect of low-rank approximation: SVD reduced the dense classifier’s degrees of freedom while preserving the main signal directions.

Table 7: Class-wise diagnostic comparison between the baseline and SVD models.

Cell Class	Baseline F1	SVD F1	Δ F1	Baseline Recall	SVD Recall
Basophil	99.59%	99.18%	−0.41%	99.17%	98.77%
Eosinophil	99.92%	99.84%	−0.08%	99.68%	100.00%
Erythroblast	98.41%	99.19%	+0.78%	98.07%	98.71%
Immature Gran.	96.28%	97.06%	+0.78%	96.70%	96.89%
Lymphocyte	98.78%	98.37%	−0.41%	98.36%	99.18%
Monocyte	97.90%	98.59%	+0.69%	95.88%	98.24%
Neutrophil	97.73%	97.98%	+0.25%	98.33%	98.20%
Platelet	100.00%	100.00%	0.00%	99.58%	100.00%

The class-wise results show that the low-rank approximation preserved minority-class boundaries. The minimum per-class F1 improved from 96.28% to 97.06% for Immature Granulocytes.

4.2.5 Statistical Efficiency vs. Hardware Limitations

SVD substantially improved statistical and storage efficiency. The active parameter count fell from 139.6 million to 21.6 million. The Akaike Information Criterion (AIC) decreased from 2.79×10^8 to 4.32×10^7 , and the Bayesian Information Criterion (BIC) decreased from 1.13×10^9 to 1.75×10^8 . The physical model size also fell from 532.5 MB to 82.42 MB, giving a compression ratio of $6.46\times$.

However, inference latency did not improve. Despite a 98.69% theoretical reduction in fully connected FLOPs, the model recorded 208.1 ± 16.6 ms/batch, slightly slower than the 195.7 ms baseline. This happened because the low-rank head split one large matrix operation into two smaller sequential operations. At very small ranks ($k = 44$ and $k = 31$), the GPU did not use its parallel cores efficiently, so kernel overhead and memory movement limited the speedup.

Thus, SVD solved the storage problem left by L_1 and improved statistical efficiency, but it did not

solve end-to-end latency. The remaining bottleneck was the convolutional base. This motivates the final method, L_0 Gaussian gating, which targets whole convolutional channels and then rebuilds the network through tensor surgery.

4.3 Structural Compression via L_0 Gaussian Gating and Tensor Surgery

The final method addresses the main limitation left by L_1 and SVD. L_1 showed weight-level redundancy, but the resulting sparsity was unstructured and did not reduce dense hardware computation. SVD reduced the size of the fully connected layers, but end-to-end latency was still limited by the convolutional base and by sequential low-rank operations. L_0 Gaussian gating was therefore used as a structured pruning method. Instead of removing isolated weights, it learns which convolutional channels can be removed and then rebuilds the network without those channels.

4.3.1 Empirical Justification for Structural Compression (PCA Audit)

Before modifying the architecture, Principal Component Analysis (PCA) was used to estimate redundancy in the hidden activation spaces of the baseline VGG19 model. Activations were extracted from 3,000 BloodMNIST samples, and the number of principal components required to retain 95% of the variance was recorded for each layer.

Table 8: PCA redundancy audit for the baseline VGG19 architecture.

Layer Name	Total Dimensions	Dimensions for 95% Variance	Redundancy
Convolutional Block 3	256	41	84.0%
Convolutional Block 4	512	122	76.2%
Convolutional Block 5	512	59	88.5%
FC0 activations	4096	285	93.0%
FC3 activations	4096	137	96.7%

The PCA audit showed clear over-parameterization. In the convolutional base, Block 3 needed only 41 of 256 dimensions, giving 84.0% redundancy. The deeper Block 5 needed only 59 of 512 dimensions, giving 88.5% redundancy. The fully connected layers were even more redundant, with redundancy above 93%. The SVD experiment had already exploited the dense-layer part of this audit. The remaining question was whether the convolutional redundancy could be converted into actual deployment speed, which led us to use L_0 Gaussian gating for structured channel removal.

4.3.2 Architectural Design: L_0 Gaussian Gates

To reduce redundancy in the convolutional base, this study used L_0 norm regularization as a direct pruning mechanism. The L_0 norm counts the number of non-zero parameters in a model, so it is a natural choice when the objective is to remove unnecessary structure rather than merely shrink weights. The main difficulty is that this count is discrete and cannot be optimized directly with standard backpropagation. To address this, the method uses Gaussian stochastic gates as a smooth approximation to the L_0 pruning objective.

We also tried to make the pruning physically useful, not only mathematically sparse. For that reason, rather than pruning single weights one by one, we used structured channel-level pruning. Each output channel in the convolutional layers received its own gate. During training, each gate was sampled from a Gaussian distribution with a trainable mean μ and a fixed standard deviation

σ , then clipped between 0 and 1. This smooth version allowed gradients to pass through the gate. During inference, the gate became a hard decision: if $\mu > 0$, the channel stayed active; if $\mu \leq 0$, the channel was turned off.

Gaussian approximation versus hard concrete Louizos et al. (2018) originally proposed the hard-concrete distribution for stochastic binary gates in L_0 regularization. In our pre-trained VGG19 setting, however, hard-concrete gates created an initialization problem: all channels needed to start open, so $\log \alpha$ had to begin far from the closing threshold. Across six pilot runs with $\log \alpha \in \{5.0, 8.0, 10.0\}$ and $\lambda \in \{1 \times 10^{-3}, 5 \times 10^{-3}, 1 \times 10^{-2}\}$, all 16 gate layers stayed at 0% sparsity through 12 epochs. The regularization needed to close gates destabilized the classifier before useful sparsity appeared.

We therefore replaced the hard-concrete gate with a Gaussian approximation. Each channel gate used a learnable location value, a noise scale, and a random Gaussian term clipped between zero and one. Initializing the gate-location parameter at 0.5 kept all channels active at epoch zero while placing them near the open/closed decision boundary. At inference time, positive gate-location values were kept and non-positive values were removed, allowing the learned sparsity pattern to be physically rebuilt through tensor surgery.

Confirmation of gate decisions To check whether the network made clear pruning decisions, we examined the trained gate parameter μ . As shown in Figure 16, the gates did not cluster around the decision boundary at $\mu = 0$. Instead, they formed a clear bimodal pattern. This shows that the gates committed to either open or closed states rather than staying uncertain.

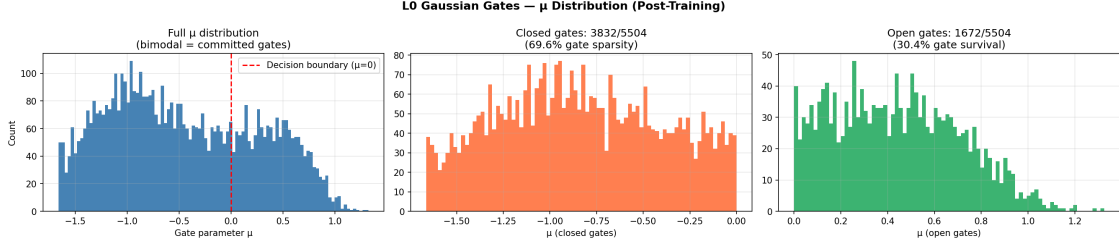


Figure 16: Post-training L_0 gate distribution. The bimodal pattern shows that gates committed to open or closed states rather than remaining near the pruning threshold.

4.3.3 Training and PID-Controlled Sparsification

After defining the architecture, we loaded the trained BloodMNIST baseline weights and trained the model for 40 epochs. To help the network keep its classification ability, we used two learning rates. We used a small learning rate of 1×10^{-5} for the convolutional weights to protect the learned features. We used a larger learning rate of 2×10^{-3} for the gate parameters so they could make clear keep-or-remove decisions. Appendix B reports the complete L_0 gate schedule and PID controller settings.

The PID-controlled regularization schedule reduced the open gate count from 5,504 to 1,572 surviving channels, corresponding to 32.1% gate survival and 69.06% gate sparsity. Once the network passed the 60% sparsity target, the controller lowered the penalty so the remaining convolutional weights

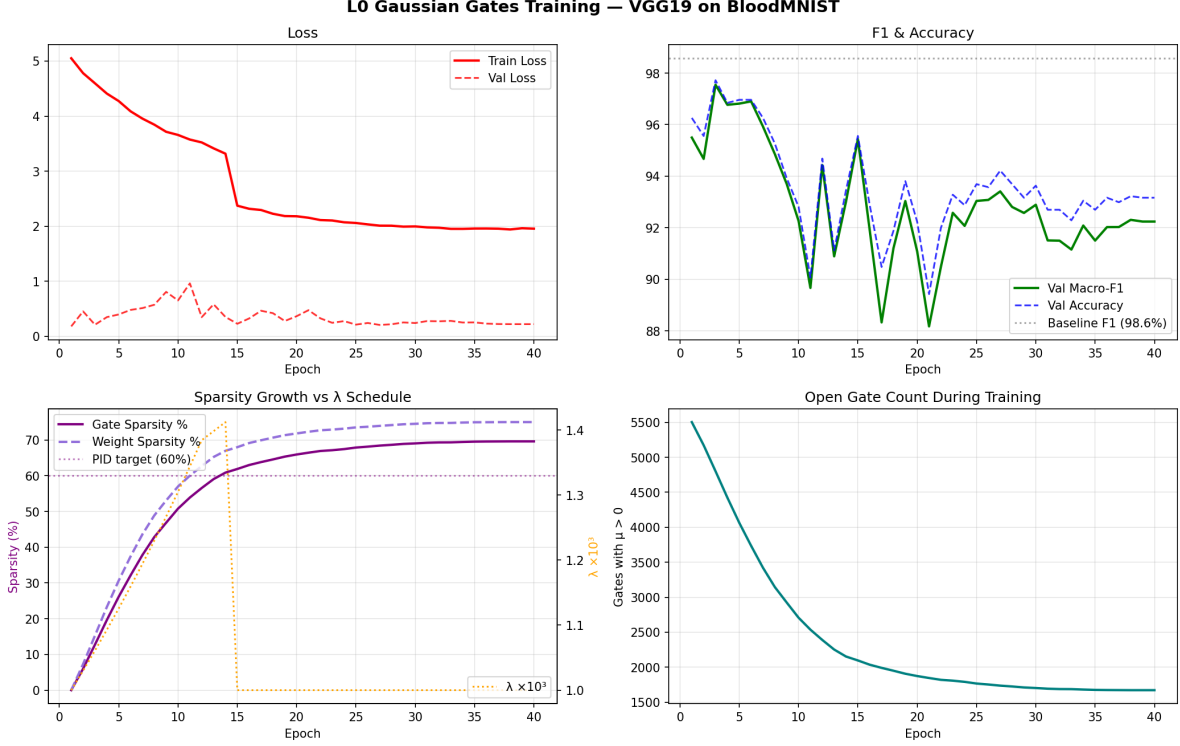


Figure 17: L0 Gaussian gate training dynamics. The curves show PID-controlled sparsification, recovery after the target is reached, and the validation trade-off used for checkpoint selection.

could recover. Appendix B reports the full epoch-by-epoch controller path (Table 29).

Epoch 30 gave the best practical checkpoint. The model reached 69.1% gate sparsity, silenced 74.5% of the convolutional weights, and kept a top-1 accuracy of 93.98% with a macro-averaged F1-score of 93.11%. Information criteria also favored the simpler structure: AIC = 71,092,301.55 and BIC = 289,258,521.04, both far below the baseline despite the expected likelihood loss.

Checkpoint selection criterion The checkpoint rule selected the highest sparsity model whose validation F1 stayed within 5% of the best observed validation F1. Under this rule, epoch 30 was retained: it reached 74.54% weight sparsity with validation F1 of 92.89%, while later epochs produced only small sparsity gains without improving validation F1.

The layer-by-layer survival pattern shows that pruning depended strongly on depth. Early convolutional Blocks 1 and 2 had the highest survival rates, ranging from 62.5% to 87.5%, suggesting that low-level visual features remained important. Convolutional Block 3 acted as a transition zone, with survival rates between 45.3% and 52.0%.

The deepest convolutional blocks, Blocks 4 and 5, were pruned the most, with survival rates between 16.2% and 24.2%. This agrees with the PCA audit. The final convolutional layer in Block 5 needed only 59 of 512 dimensions to capture 95% of its activation variance, and L0 gate training reached a similar result by keeping only 83 of 512 channels in that layer. The agreement between PCA and L0 training supports our view that many deep VGG19 features are unnecessary for this 8-class

Table 9: Per-layer gate survival after L0 training in the VGG19 convolutional base.

Layer	Block	Total Channels	Open Channels	Closed Channels	Survival
1	Convolutional Block 1	64	40	24	62.5%
2	Convolutional Block 1	64	47	17	73.4%
3	Convolutional Block 2	128	97	31	75.8%
4	Convolutional Block 2	128	112	16	87.5%
5	Convolutional Block 3	256	122	134	47.7%
6	Convolutional Block 3	256	116	140	45.3%
7	Convolutional Block 3	256	127	129	49.6%
8	Convolutional Block 4	256	133	123	52.0%
9	Convolutional Block 4	512	115	397	22.5%
10	Convolutional Block 4	512	112	400	21.9%
11	Convolutional Block 4	512	123	389	24.0%
12	Convolutional Block 4	512	124	388	24.2%
13	Convolutional Block 5	512	114	398	22.3%
14	Convolutional Block 5	512	97	415	18.9%
15	Convolutional Block 5	512	110	402	21.5%
16	Convolutional Block 5	512	83	429	16.2%
Total		4896	1572	3324	32.1% open / 67.9% pruned

blood-cell task, even though they may be useful for the original 1,000-class ImageNet task.

4.3.4 Masked Sparsity and Physical Tensor Surgery

Although the model achieved a theoretical convolutional FLOP reduction of 74.5%, the first masked L0 model was actually slower. Its inference latency increased to 241.4 ms/batch, which was 45.7 ms slower than the baseline. This happened because L0 training first creates what we call ghost sparsity. The network masks discarded channels by multiplying them by zero, but those channels still remain in GPU memory, and the hardware still spends time processing zeroed outputs and binary masks.

To turn this compression into real hardware speedup, we tried to fix the ghost-sparsity problem by applying a post-training tensor-surgery algorithm. The algorithm scanned the trained masked model, found the surviving convolutional channels with gate values above zero, and built a smaller network called Dense Pruned VGG. Appendix C provides the implementation details for this physical reconstruction step.

For each convolutional layer, the script physically sliced the weight and bias tensors. It kept only the surviving channels and deleted the inactive ones. We also added a mapping step to connect the final convolutional output to the first fully connected layer. By identifying which flattened spatial indices belonged to removed channels, the algorithm resized the first linear layer correctly and kept the tensors aligned.

To check whether this fix preserved the original computation, we passed a random diagnostic tensor through both the original ghost-sparse model and the rebuilt physical model. The maximum absolute output difference was only 1.9×10^{-6} , which is within floating-point precision. This confirmed that the physical extraction preserved the model’s mapping.

4.3.5 Diagnostic Performance After Structured Pruning

At this sparse state, the network reached 93.98% top-1 accuracy and 93.11% macro-F1. Clear and well-represented classes, such as Platelets, Eosinophils, and Erythroblasts, kept F1-scores above 98%, suggesting that the surviving convolutional channels preserved their main visual features.

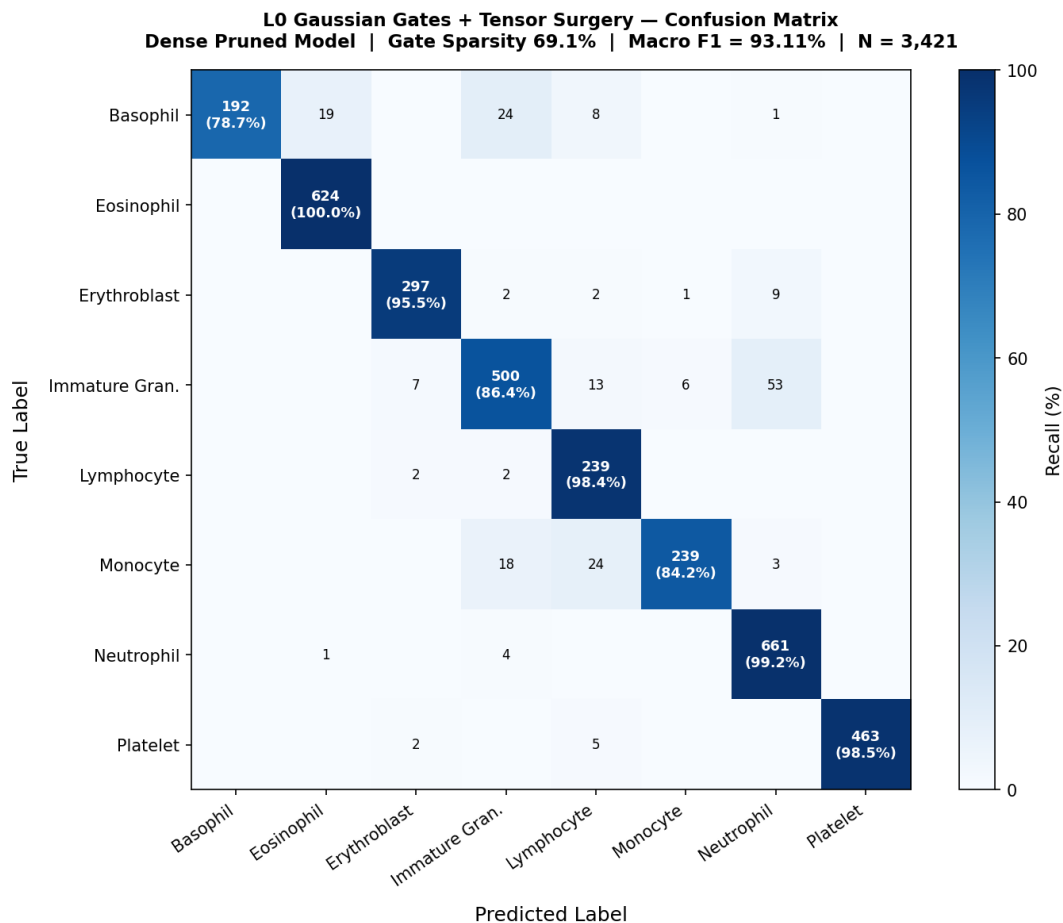


Figure 18: Confusion matrix for the L0 Gaussian gate model after tensor surgery. Most predictions remain diagonal, while errors concentrate in the classes most sensitive to channel removal.

Figure 18 shows that most predictions remain concentrated on the diagonal, while the main errors occur in morphologically similar or minority classes. The largest losses appeared for Basophils (88.07% F1) and Immature Granulocytes (88.57% F1), suggesting that some pruned deep features still helped the baseline separate subtle blood-cell differences.

4.3.6 Hardware Deployment Metrics

The physical reconstruction produced a clear three-stage size pattern. The uncompressed baseline VGG19 used 532.5 MB on disk. The trained masked model used 532.58 MB because it still kept the full original tensors plus the gate parameters. This shows that L0 gate training adds almost no

storage cost during training, but it also does not shrink the model by itself. After tensor surgery removed inactive channels and weights, the deployable model shrank to 134.88 MB. This gave a physical compression ratio of $3.95\times$ compared with the uncompressed baseline.

However, active memory did not fall as much as disk size. Disk storage and theoretical FLOPs followed the 74.5% parameter reduction closely, but inference memory changed only slightly. Memory decreased from 5,632.7 MB in the masked model to 5,572.4 MB in the dense pruned model. This suggests that VRAM use during inference was driven more by activation maps and fixed batch size than by the stored weight tensors.

Even with this memory plateau, the physically rebuilt model avoided the extra work caused by binary masking. Under the same hardware conditions, the physically pruned L0 network reached an inference latency of 96.9 ± 7.7 ms/batch. This was 50.5% faster than the baseline and 59.9% faster than the masked model. This result suggests that structured L_0 pruning can produce real inference speedup when it is followed by tensor surgery.

Table 10: Complete results summary for the L0 Gaussian gate model.

Metric	Value	Comparison
Gate Sparsity	69.06%	—
Conv Weight Sparsity	74.54%	—
Active Parameters	$\sim 35.6\text{M}$ (conv survivors + full FC)	139.6M baseline
Model Size — Masked	532.58 MB	$\sim$ baseline
Model Size — Dense (post-surgery)	134.88 MB	−74.7%
Physical Compression Ratio	$3.95\times$	—
Top-1 Accuracy	93.98%	—
Macro F1-Score	93.11%	−5.46%
Macro ROC-AUC	99.68%	−0.26%
Matthews Correlation Coefficient (MCC)	93.00%	−5.23%
Cohen’s Kappa (κ)	92.95%	−5.28%
Mean Test Loss	0.2367	+0.1879
Min Per-Class F1	88.07% (Basophil)	—
Latency — Masked Model	241.4 ± 18.1 ms/batch	+45.7 ms vs. baseline
Latency — Dense Pruned Model	96.9 ± 7.7 ms/batch	−50.5% vs. baseline
Memory Footprint — Masked	5,632.7 MB	—
Memory Footprint — Dense	5,572.4 MB	−1.1% vs. masked
Conv FLOP Reduction (theoretical)	74.54%	vs. baseline
Log-Likelihood	−809.78	vs. baseline −166.85
Likelihood Ratio vs. Baseline	−1,285.86	—
AIC	71,092,301.55	vs. baseline 279,206,365.70
BIC	289,258,521.04	vs. baseline 1,136,046,147.59

The Matthews Correlation Coefficient (MCC) was 93.00%, and Cohen’s Kappa (κ) was 92.95%. These losses are consistent with the macro-F1 drop and suggest moderate class-level degradation rather than collapse in a single class.

The per-class analysis shows that performance did not drop equally across classes. Visually distinct

Table 11: Per-class classification performance of L0 Gaussian gates relative to the baseline VGG19 model.

Cell Class	Support	F1	Precision	Recall	Specificity	AUC	Δ F1 vs. Baseline
Basophil	244	88.07%	100.00%	78.69%	100.00%	99.84%	−11.52%
Eosinophil	624	98.42%	96.89%	100.00%	99.28%	99.98%	−1.50%
Erythroblast	311	95.96%	96.43%	95.50%	99.65%	99.53%	−3.54%
Immature Gran.	579	88.57%	90.91%	86.36%	98.24%	99.01%	−8.00%
Lymphocyte	243	89.51%	82.13%	98.35%	98.36%	99.81%	−9.90%
Monocyte	284	90.19%	97.15%	84.15%	99.78%	99.47%	−8.40%
Neutrophil	666	94.90%	90.92%	99.25%	97.60%	99.79%	−3.60%
Platelet	470	99.25%	100.00%	98.51%	100.00%	100.00%	−0.75%
Macro Average	3,421	93.11%	94.43%	92.60%	99.11%	99.68%	−5.46%

classes such as Platelet, Eosinophil, Erythroblast, and Neutrophil had smaller F1 losses, while classes requiring subtler high-level discrimination were more sensitive to deep-layer pruning. Basophil had the largest F1 decline at 11.52%, and Lymphocyte and Immature Granulocytes followed a similar pattern. Statistically, this pattern suggests that L_0 pruning mainly removes shared convolutional capacity; classes depending on fine-grained morphology lose more information, while classes with distinctive visual signatures remain stable. Even so, all eight classes kept F1-scores above 88%, and no class fell below the 85% interpretive guardrail set at the start of training.

Chapter 5

Discussion and Unified Methodological Comparison

5 Discussion and Unified Methodological Comparison

5.1 Comparison of Compression Methods

This chapter compares the three compression methods under one statistical and computational framework. Each method tests a different form of redundancy: L_1 Lasso targets individual weights, SVD targets low-rank structure in the dense classifier, and L_0 Gaussian gating targets removable convolutional channels.

The results show a clear trade-off between predictive performance, statistical parsimony, physical model size, and latency. Figure 19 places the methods along this trade-off, and Figure 20 gives the complementary predictive view. SVD has the strongest observed aggregate predictive metrics, while the L_0 dense model gives up some macro-F1 in exchange for the largest observed latency improvement.

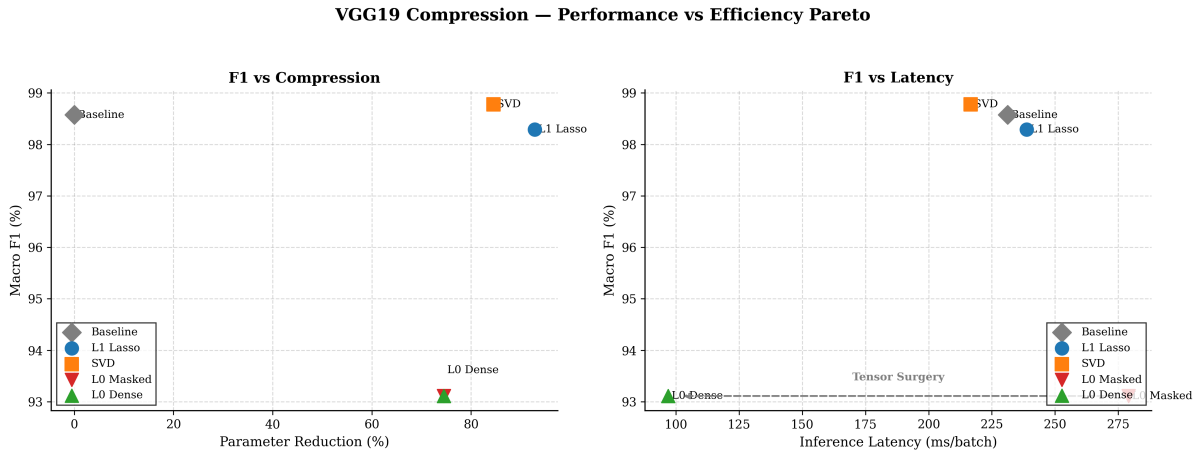


Figure 19: Unified Pareto comparison of compression methods. The plot contrasts predictive performance against compression and latency, showing that the best statistical model is not necessarily the fastest deployable model.

This unified comparison supports the broader edge-AI view that compression should be judged by accuracy, memory, and latency together, not by parameter count alone (Francy & Singh, 2024). No method is best under every criterion in these experiments: L_1 gives the largest nominal parameter reduction, SVD gives the strongest observed predictive result, and the dense L_0 model gives the strongest observed deployment result after tensor surgery.

Tables 12 and 13 compare the methods across predictive, structural, and hardware criteria. Table 14 adds the statistical model-selection view. Together, these tables show that statistical parsimony, physical compression, and realized speed are related but not identical. Appendix A documents the hardware and software environment used for these measurements.

Table 12 shows that L_1 and SVD preserve baseline-level diagnostic performance, while L_0 sacrifices about 5.46 percentage points of macro-F1. The identical predictive scores for masked and dense L_0 confirm that tensor surgery changes deployment structure rather than the learned decision rule.

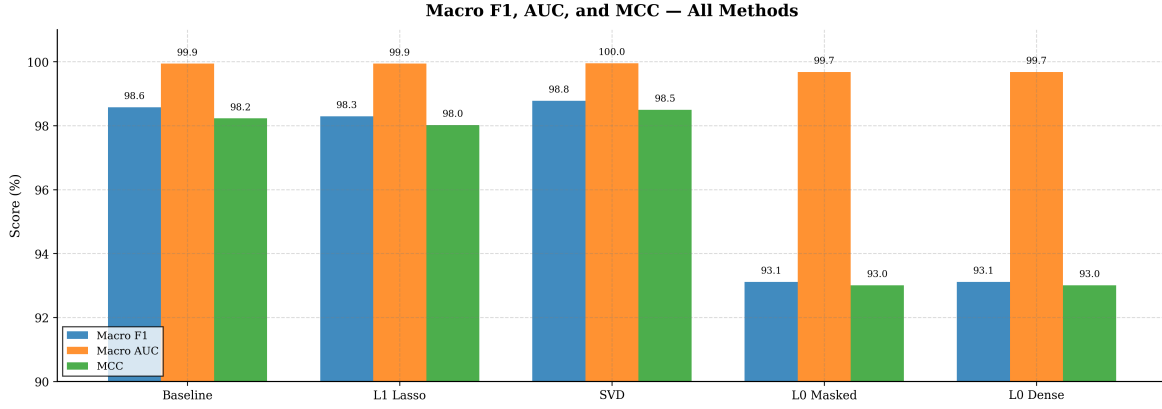


Figure 20: Grouped predictive-metric comparison. SVD preserves or improves aggregate predictive metrics in these experiments, while the dense L_0 model trades some F1 for observed hardware gains.

Table 12: Unified performance metrics across all compression paradigms.

Method	Top-1 Acc	Macro F1	Δ F1	AUC	MCC	Kappa	Test Loss	Min-Class F1
Baseline	98.48%	98.57%	—	99.94%	98.23%	98.23%	0.0488	96.28%
L_1 Lasso	98.30%	98.29%	−0.29%	99.94%	98.02%	98.02%	0.0565	96.36%
SVD	98.71%	98.77%	+0.20%	99.95%	98.50%	98.50%	0.0404	97.06%
L_0 Masked	93.98%	93.11%	−5.46%	99.68%	93.00%	92.95%	0.2367	88.07%
L_0 Dense	93.98%	93.11%	−5.46%	99.68%	93.00%	92.95%	0.2367	88.07%

Table 13: Structural compression and hardware efficiency metrics.

Method	Active Params	Reduction	Model Size	Memory	Latency (ms/batch)	Speedup
Baseline	139.60 M	0.0%	532.6 MB	3771.7 MB	231.3 ms	1.000×
L_1 Lasso	10.00 M	92.8%	532.6 MB	3771.7 MB	238.8 ms	0.969×
SVD	21.60 M	84.5%	82.4 MB	3321.5 MB	216.6 ms	1.068×
L_0 Masked	35.54 M	74.5%	532.6 MB	4555.5 MB	279.1 ms	0.829×
L_0 Dense	35.54 M	74.5%	134.9 MB	5572.4 MB	96.9 ms	2.387×

Table 13 separates nominal compression from realized speed. L_1 removes the largest share of active weights but does not reduce dense tensor dimensions, whereas dense L_0 gives the strongest latency gain because tensor surgery physically changes the convolutional architecture.

Table 14: Information-theoretic and statistical metrics.

Method	Log-Likelihood	AIC	BIC	LR vs. Baseline
Baseline	-166.85	279,206,365.70	1,136,046,147.59	<i>Reference</i>
L_1 Lasso	-193.15	20,000,844.29	81,379,131.67	-52.60
SVD	-138.27	43,207,076.53	175,802,009.47	+57.16
L_0 Dense	-809.78	71,089,499.55	289,247,120.14	-1285.86

Table 14 shows that all compressed models reduce AIC and BIC because the parameter penalty falls sharply. SVD is the only method with a positive likelihood-ratio contrast against the baseline, indicating that its low-rank constraint improved fit as well as reducing complexity.

5.2 Hardware Efficiency and Structural Compression

The unified results show an important distinction: reducing parameter count is not the same as reducing inference time. Figure 21 summarizes the statistical model-selection side of this comparison, while the latency results show that parsimony alone does not guarantee faster execution. Hardware gains depend on whether compression changes the tensor operations that actually run, a distinction also emphasized by efficient-CNN design work and deployment-oriented compression studies (Howard et al., 2017; Tan & Le, 2019; Francy & Singh, 2024).

L_1 Lasso and unstructured sparsity The L_1 model provides strong evidence of weight-level redundancy: its AIC fell from 2.79×10^8 to 2.00×10^7 . However, individual zero weights do not change tensor dimensions, so dense hardware still processes dense arrays. Latency therefore increased slightly despite the large active-parameter reduction.

SVD and low-rank factorization SVD gave the strongest predictive and storage result, reducing file size from 532.6 MB to 82.4 MB while increasing macro-F1 to 98.77%. The hardware gain was smaller because one large matrix multiplication became two smaller sequential operations, so kernel overhead and memory movement limited the wall-clock speedup.

L_0 Gaussian gates and tensor surgery The L_0 approach gave the clearest deployment benefit only after tensor surgery. The masked model was slower than the baseline because inactive channels were still present and multiplied by binary masks. After tensor surgery physically removed those channels, the dense pruned model shrank to 134.9 MB and reached 96.9 ms/batch, giving the strongest latency result. Appendix C details the tensor-surgery procedure that converted ghost sparsity into a deployable dense-pruned network.

5.3 Class-Level and Architectural Interpretations

The L_0 pruning pattern also gives information about how VGG19 represents BloodMNIST images. Early convolutional layers were mostly preserved, while deeper layers were pruned more heavily. This

Information Criteria — AIC and BIC
(Lower = better fit per parameter complexity)

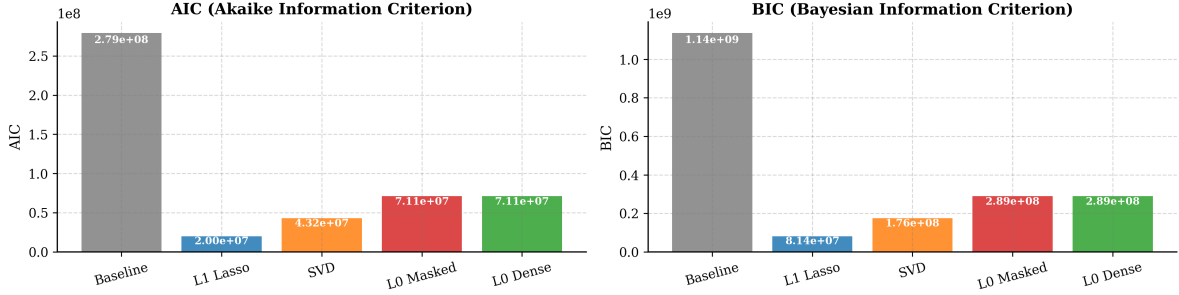


Figure 21: AIC and BIC comparison across models. The large reductions relative to baseline show how compression changes the statistical complexity penalty, not only the deployed file size.

suggests that low-level visual features remain broadly useful, while many deeper ImageNet-specific features are redundant for this eight-class blood-cell task.

Figure 22 shows that the diagnostic loss from compression is not uniform across classes. Visually distinctive classes remain relatively stable, while morphologically ambiguous classes are more sensitive to channel removal.

The per-class results support this interpretation: visually distinctive classes remained stable, while larger losses were concentrated in Basophils and Immature Granulocytes, which require finer morphological distinctions.

Even with uneven class-level degradation, all eight classes in the dense L_0 model remained above the 85.0% interpretive guardrail. This threshold is used as an internal research guardrail rather than a clinical-approval standard, since medical AI performance must be judged with external validation, workflow integration, and clinical governance rather than accuracy alone (Kelly et al., 2019). Overall, the results suggest that statistically guided compression can remove substantial architectural excess, but structured compression is needed when latency is the main deployment goal.

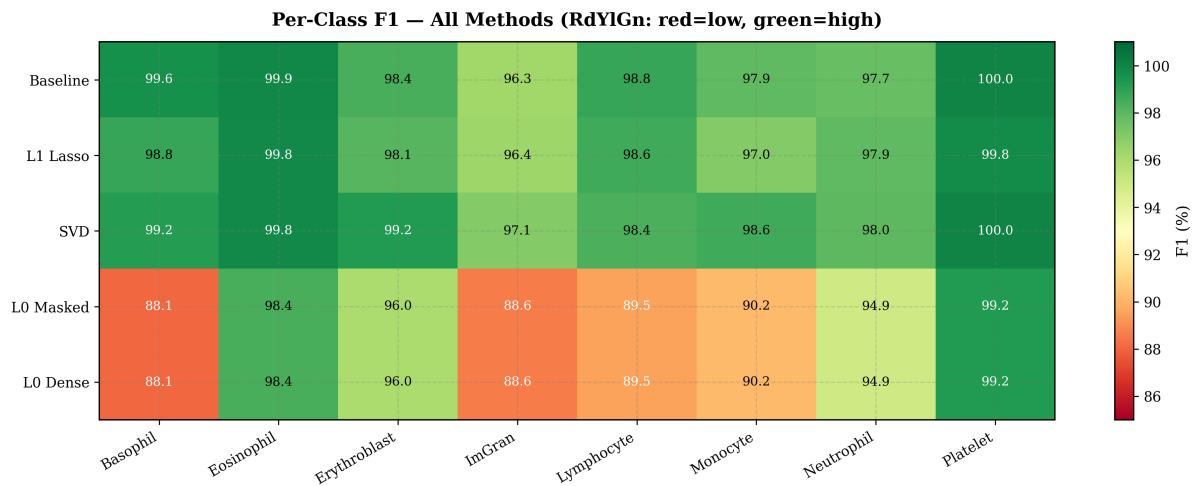


Figure 22: Per-class F1-score heatmap across compression methods. The heatmap highlights that compression loss is class-dependent, with visually ambiguous classes degrading more than visually distinctive ones.

Chapter 6

Conclusions and Recommendations

6 Conclusions and Recommendations

6.1 Summary of Key Findings

This study asked whether statistical compression methods can reduce the complexity of an over-parameterized VGG19 model while preserving useful BloodMNIST classification performance. The results support this goal, but they also show that each method solves a different part of the problem. Compression should therefore be judged by several criteria at once: prediction, statistical parsimony, model size, memory use, and latency.

Three findings are most important. First, L_1 Lasso regularization showed substantial weight-level redundancy, reaching 92.8% unstructured sparsity, but did not improve inference speed because tensor shapes were unchanged. Second, Singular Value Decomposition (SVD) gave the strongest observed accuracy-preserving result, improving macro-F1 to 98.77% and reducing model size to 82.4 MB. Third, the Gaussian relaxation of the L_0 norm gave the strongest observed deployment result when combined with tensor surgery, reducing model size to 134.88 MB and cutting inference latency by 50.5%.

Overall, no single compression method was best under every criterion in these experiments. SVD appears most suitable when predictive performance is the main objective. Structured L_0 pruning with tensor surgery appears most suitable when inference speed is the main deployment objective. L_1 regularization is most useful as a statistical diagnostic for identifying over-parameterization.

6.2 Methodological Contributions

This study makes three contributions to applied statistics and deep learning. The first is conceptual, the second is empirical, and the third is methodological and deployment-oriented. A central contribution is the Gaussian-gated L_0 pruning pipeline combined with tensor surgery.

First, the study frames neural-network compression as a statistical model-selection problem rather than only an engineering optimization problem. This framing is supported by statistical views of deep learning and sparse model reduction, where over-parameterized networks can be interpreted through generalization, redundancy, and effective complexity (Bartlett et al., 2021; Nalisnick, Smyth, & Tran, 2023; Bai, Song, & Cheng, 2020). In this view, sparsity, low-rank approximation, and information criteria are used as statistical evidence about redundancy, parsimony, and model complexity.

Second, the study provides a controlled empirical comparison of L_1 regularization, SVD compression, and structured L_0 pruning in the same diagnostic imaging setting. This comparison contributes to the broader compression literature, which shows that pruning, quantization, low-rank approximation, and other compression methods can have different effects on accuracy, memory, and latency (Blalock et al., 2020; Kuzmin et al., 2021; Francy & Singh, 2024; Liu et al., 2025). The results separate three outcomes that are often treated as interchangeable: statistical sparsity, physical model-size reduction, and realized hardware acceleration.

Third, the study develops and evaluates a deployable Gaussian-gated L_0 pruning pipeline for

structured channel selection. Prior neural-network L_0 pruning work commonly uses hard-concrete, binary, or other non-Gaussian gate relaxations to make the discrete sparsity objective trainable (Louizos, Welling, & Kingma, 2018; Yamamoto & Maeno, 2018; Oliveira, Batista, & Seara, 2023). This study instead uses a Gaussian relaxation to learn channel gates. Tensor surgery then converts those learned masks into a physically smaller dense VGG19 architecture by removing zero-gated channels and remapping the downstream tensors. This makes the contribution more than a comparison of existing compression tools: it connects a statistical gate-learning method to a deployable dense reconstruction procedure and shows that the latency benefit appears only after inactive channels are removed from the computation graph.

6.3 Recommendations for Practitioners

For researchers and machine learning engineers evaluating diagnostic-imaging models in resource-constrained environments, the results suggest three practical recommendations:

1. **Avoid unstructured sparsity on standard dense hardware** Magnitude-based or L_1 -regularized unstructured pruning should not be expected to improve inference speed on standard GPUs or CPUs unless the deployment environment explicitly supports sparse Basic Linear Algebra Subprograms (BLAS).
2. **Match the compression method to the source of redundancy** SVD is well suited to fully connected layers where redundancy is mainly linear and low rank. Structured L_0 pruning is better suited to convolutional layers where redundant channels can be removed physically.
3. **Use tensor surgery when inference latency matters** Structured sparsity alone is not enough if pruned channels remain in the execution graph. To obtain practical latency benefits, the trained sparse model should be converted into a smaller dense model with correctly remapped tensors.

6.4 Recommendations for Future Research

The results suggest several directions for future work, especially around combining compression methods and extending the statistical framework.

Combining L_0 pruning and SVD This study evaluated L_0 Gaussian gates and SVD separately so their individual effects could be measured. Future work should test a combined pipeline in which L_0 tensor surgery is applied first and SVD is then applied to the remaining dense layers, targeting both convolutional and classifier redundancy in one model.

Bayesian compression and hardware limits The L_0 relaxation used in this study produced useful sparsity, but Bayesian variable-selection methods such as Spike-and-Slab priors provide a more formal probabilistic framework. Related Bayesian compression work suggests that posterior sparsity can support compact uncertainty-aware models (Saha, Liu, & Liang, 2024). Future work should investigate scalable Bayesian approximations that preserve these statistical benefits without exceeding standard hardware limits.

Quantization and edge deployment This study focused on pruning and factorization with 32-bit floating-point weights. Future work should test mixed pipelines that combine structured pruning, SVD, and post-training or quantization-aware compression, while monitoring whether class-level performance remains stable for minority or morphologically ambiguous classes (Zhao, Yue, & Hu, 2020; Lang, Guo, & Huang, 2024).

Extensions to Transformer architectures The same statistical ideas may also apply to Transformer models, which contain dense projection matrices in attention and feed-forward blocks. Future research should examine whether low-rank adaptation, quantization, and structured sparsity can be combined for deployment on resource-constrained hardware.

6.5 Conclusion

The main conclusion is that statistical compression can substantially reduce neural-network complexity, but the compression method must match the type of redundancy being removed. L_1 regularization was most useful for detecting weight-level redundancy, SVD was most useful for preserving predictive performance while shrinking dense layers, and structured L_0 pruning with tensor surgery was most useful for reducing inference latency. More broadly, compression should be treated as a criterion-specific model-selection problem whose most suitable method depends on the objective: statistical parsimony, predictive fidelity, storage reduction, or latency.

All experiments in this study were implemented in Python using PyTorch and related scientific computing libraries. The appendix materials provide the hardware environment, hyperparameter schedules, and tensor-surgery reference implementation needed to reproduce the reported results.

References

1. Åström, K. J., & Hägglund, T. (1995). *PID controllers: Theory, design, and tuning* (2nd ed.). Instrument Society of America.
2. Acharyya, R., Chattoraj, A., Zhang, B., Das, S., & Štefankovič, D. (2021). *Statistical mechanical analysis of neural network pruning*. University of Rochester.
3. Aaronson, S., Lee, L. L., & Li, J. (2026). Sparsity and out-of-distribution generalization. arXiv preprint arXiv:2603.07388.
4. Mohamad, M. Y., Bevrani, H., & Haydari, A. A. (2024). *Statistical tuning of artificial neural network*. arXiv preprint arXiv:2409.16426.
5. Bai, J., Song, Q., & Cheng, G. (2020). Efficient variational inference for sparse deep learning with theoretical guarantee. *Advances in Neural Information Processing Systems*, 33, 9974–9985.
6. Bartlett, P. L., Montanari, A., & Rakhlin, A. (2021). Deep learning: A statistical viewpoint. *Acta Numerica*, 30, 87–201.
7. Bellman, R. E. (1957). *Dynamic programming*. Princeton University Press.
8. Blalock, D., Gonzalez Ortiz, J. J., Frankle, J., & Gutttag, J. (2020). What is the state of neural network pruning? *Proceedings of Machine Learning and Systems*, 2, 129–146.
9. Breiman, L. (2001). Statistical modeling: The two cultures. *Statistical Science*, 16(3), 199–231.
10. Burnham, K. P., & Anderson, D. R. (2002). *Model selection and multimodel inference: A practical information-theoretic approach* (2nd ed.). Springer.
11. Chatzikonstantinou, C., Papadopoulos, G. T., Dimitropoulos, K., & Daras, P. (2020). Neural network compression using higher-order statistics and auxiliary reconstruction losses. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops* (pp. 716–717).
12. Eckart, C., & Young, G. (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, 1, 211–218.
13. Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874.
14. Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., & Wichmann, F. A. (2020). Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2, 665–673.
15. Francy, S., & Singh, R. (2024). Edge AI: Evaluation of model compression techniques for convolutional neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 35, 8924–8938.
16. Frantar, E., & Alistarh, D. (2022). Optimal brain compression: A framework for accurate post-training quantization and pruning. *Advances in Neural Information Processing Systems*.

17. Frankle, J., & Carbin, M. (2019). The lottery ticket hypothesis: Finding sparse, trainable neural networks. *International Conference on Learning Representations*.
18. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
19. Gou, J., Yu, B., Maybank, S. J., & Tao, D. (2021). Knowledge distillation: A survey. *International Journal of Computer Vision*, 129, 1789–1819.
20. Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the International Conference on Machine Learning* (pp. 1321–1330).
21. Hassibi, B., & Stork, D. G. (1993). Second order derivatives for network pruning: Optimal Brain Surgeon. In *Advances in Neural Information Processing Systems* (pp. 164–171).
22. Han, S., Mao, H., & Dally, W. J. (2016). Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. In *International Conference on Learning Representations*.
23. Hinton, G., Vinyals, O., & Dean, J. (2015). *Distilling the knowledge in a neural network*. arXiv preprint arXiv:1503.02531.
24. Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). *MobileNets: Efficient convolutional neural networks for mobile vision applications*. arXiv preprint arXiv:1704.04861.
25. Jolliffe, I. T. (2002). *Principal component analysis* (2nd ed.). Springer.
26. Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). *Scaling laws for neural language models*. arXiv preprint arXiv:2001.08361. <https://doi.org/10.48550/arXiv.2001.08361>.
27. Kelly, C. J., Karthikesalingam, A., Suleyman, M., Corrado, G., & King, D. (2019). Key challenges for delivering clinical impact with artificial intelligence. *BMC Medicine*, 17, 195.
28. Lawson, C. L., Hanson, R. J., Kincaid, D. R., & Krogh, F. T. (1979). Basic Linear Algebra Subprograms for Fortran usage. *ACM Transactions on Mathematical Software*, 5(3), 308–323.
29. Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *Proceedings of the International Conference on Learning Representations*.
30. Kuzmin, A., Nagel, M., van Baalen, M., Behboodi, A., & Blankevoort, T. (2021). *Pruning vs. quantization: Which is better?* arXiv preprint arXiv:2102.04544.
31. Lang, J., Guo, Z., & Huang, S. (2024). *A comprehensive study on quantization techniques for large language models*. arXiv preprint arXiv:2308.13137.
32. LeCun, Y., Denker, J. S., & Solla, S. A. (1990). Optimal brain damage. In *Advances in Neural Information Processing Systems* (pp. 598–605).
33. Loshchilov, I., & Hutter, F. (2017). SGDR: Stochastic gradient descent with warm restarts. In *Proceedings of the International Conference on Learning Representations*.

34. Liu, D., Zhu, Y., Liu, Z., Liu, Y., Han, C., Tian, J., Li, R., & Yi, W. (2025). A survey of model compression techniques: Past, present, and future. *Frontiers in Robotics and AI*, 12, 1518965. <https://doi.org/10.3389/frobt.2025.1518965>.
35. Lynch, E. C. (1990). Peripheral blood smear. In H. K. Walker, W. D. Hall, & J. W. Hurst (Eds.), *Clinical Methods: The History, Physical, and Laboratory Examinations* (3rd ed., Chapter 155). Butterworths.
36. Matthews, B. W. (1975). Comparison of the predicted and observed secondary structure of T4 phage lysozyme. *Biochimica et Biophysica Acta*, 405(2), 442–451.
37. Louizos, C., Welling, M., & Kingma, D. P. (2018). Learning sparse neural networks through L_0 regularization. *International Conference on Learning Representations*.
38. Min, J., Song, X., Zheng, S., King, C. B., Deng, X., & Hong, Y. (2024). Applied statistics in the era of artificial intelligence: A review and vision. *Statistics and Its Interface*, 17, 123–145.
39. Mirsky, L. (1960). Symmetric gauge functions and unitarily invariant norms. *Quarterly Journal of Mathematics*, 11(1), 50–59.
40. Nalisnick, E., Smyth, P., & Tran, D. (2023). A brief tour of deep learning from a statistical perspective. *Annual Review of Statistics and Its Application*, 10, 219–246.
41. Nesterov, Y. E. (1983). A method for solving the convex programming problem with convergence rate $O(1/k^2)$. *Doklady Akademii Nauk SSSR*, 269(3), 543–547.
42. Oliveira, F. D. R., Batista, E. L. O., & Seara, R. (2023). On the compression of neural networks using l_0 -norm regularization and weight pruning. *Neural Networks*.
43. Powers, D. M. W. (2011). Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation. *Journal of Machine Learning Technologies*, 2(1), 37–63.
44. Rissanen, J. (1978). Modeling by shortest data description. *Automatica*, 14(5), 465–471.
45. Saha, D., Liu, Z., & Liang, F. (2024). Efficient model compression for Bayesian neural networks. *Bayesian Analysis*, 19, 447–474.
46. Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the International Conference on Machine Learning* (pp. 6105–6114).
47. Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. In *Proceedings of the International Conference on Learning Representations*.
48. Tibshirani, R. (1996). Regression shrinkage and selection via the Lasso. *Journal of the Royal Statistical Society: Series B*, 58(1), 267–288.
49. Vapnik, V. N., & Chervonenkis, A. Y. (1971). On the uniform convergence of relative frequencies of events to their probabilities. *Theory of Probability and Its Applications*, 16(2), 264–280.
50. Wu, S., Li, G., Chen, F., & Shi, L. (2018). Training and inference with integers in deep neural networks. *International Conference on Learning Representations*.

51. Wu, S., Yang, S., & Kou, S. C. (2025). *Are statistical methods obsolete in the era of deep learning?* arXiv preprint arXiv:2505.21723.
52. Xu, S., & Yu, Z. (2025). Neural networks perform sufficient dimension reduction. In *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 39, No. 20, pp. 21806–21814).
53. Yang, J., Shi, R., Wei, D., Liu, Z., Zhao, L., Ke, B., Pfister, H., & Ni, B. (2023). MedMNIST v2: A large-scale lightweight benchmark for 2D and 3D biomedical image classification. *Scientific Data*, 10, 41. <https://doi.org/10.1038/s41597-022-01721-8>.
54. Yamamoto, K., & Maeno, K. (2018). On the compression of neural networks using l_0 norm regularization and weight pruning. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing* (pp. 2832–2836).
55. Zhao, S., Yue, T., & Hu, X. (2020). Distribution-aware adaptive multi-bit quantization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 9281–9290).

A Appendix A: Hardware and Software Environment Specifications

To support transparency, reproducibility, and empirical traceability, all computational experiments were conducted under a controlled hardware and software environment.

A.1 Hardware Infrastructure

All phases of the experimental pipeline—including baseline optimization, L_1 Lasso regularization, Singular Value Decomposition (SVD), and L_0 Gaussian gate training—were executed using cloud-based computational infrastructure provided by Google Colaboratory (Colab). The specific runtime hardware allocations are summarized below:

Table 15: Hardware environment used for all computational experiments.

Component	Specification
Graphics Processing Unit (GPU)	NVIDIA T4 Tensor Core GPU
Video Random Access Memory (VRAM)	16 GB GDDR6
Compute Capability	7.5
Central Processing Unit (CPU)	Intel Xeon Processor @ 2.20 GHz (2 vCPUs)
System Memory (RAM)	12.7 GB
Storage Backend	Google Drive mounted network storage, used for checkpoint serialization and retrieval

The use of the NVIDIA T4 GPU shows that the physical inference accelerations observed in this study—for example, the 96.9 ms/batch latency of the Dense Pruned L_0 model—were measured on a standard, widely accessible GPU rather than a specialized high-performance computing cluster.

A.2 Software, AI Tools, and Libraries

The codebase was developed and executed using the software stack summarized below. In addition, generative AI tools were used in a limited supporting role for code drafting, code refactoring, paper discovery, and LaTeX conversion. Final modeling decisions, experimental design, and reported results remained the responsibility of the authors.

Table 16: Software environment and framework versions.

Component	Specification
Operating System	Ubuntu 22.04 LTS (via Colab container)
Language	Python 3.10.x
Deep Learning Framework	PyTorch 2.x compiled with CUDA 12.x support
Hardware Acceleration API	NVIDIA cuDNN
Dataset API	medmnist v3.0.2

This software stack documents the main sources of computational variation: operating system,

Python version, PyTorch/CUDA backend, and dataset API. As a result, differences reported across compression methods are interpreted mainly in relation to model structure rather than changes in the execution environment.

Table 17: AI tools used during the research and writing workflow.

Tool	Primary use in this study
Claude	Code generation support during implementation and experimentation
Google AI Studio	Code refactoring support and discussion of intermediate results
Perplexity Search	Paper discovery and literature collection
Prism	LaTeX conversion, editing workflow support, and document preparation assistance

The AI tools listed here were used as workflow aids, not as autonomous sources of experimental results. Numerical findings, model checkpoints, and interpretations were retained only when they could be traced back to executed code, saved logs, or manually reviewed LaTeX content.

Table 18: Primary Python libraries used in the experimental pipeline.

Library	Primary role
<code>pandas</code>	Tabular data handling and experiment log aggregation
<code>numpy</code>	Numerical array operations and scientific computing utilities
<code>torch</code> (PyTorch)	Model definition, optimization, and GPU-accelerated training
<code>tqdm</code>	Progress-bar tracking for iterative experiments
<code>os</code>	File-system and environment-variable management
<code>time</code>	Runtime and latency measurement utilities
<code>gc</code>	Manual garbage collection and memory cleanup
<code>matplotlib</code>	Core figure generation and diagnostic plotting
<code>seaborn</code>	Statistical visualization and presentation-ready plots
<code>scikit-learn</code>	Evaluation metrics, confusion matrices, and auxiliary analysis utilities

These libraries define the reproducibility boundary of the analysis pipeline. PyTorch determines model optimization and GPU execution, while NumPy, pandas, scikit-learn, matplotlib, and seaborn support measurement, aggregation, and visualization without changing the trained model itself.

To evaluate the VGG19 architecture at its native ImageNet resolution, the high-resolution BloodMNIST+ variant was used. The dataset was accessed programmatically with `BloodMNIST(size=224)`. This interface natively serves 224×224 image tensors and avoids the interpolation artifacts that would arise from manually resizing standard 28×28 MedMNIST images with `torchvision.transforms`.

A.3 Reproducibility Controls

Because this study relies heavily on stochastic neural-network training and continuous probability distributions—including the Gaussian relaxation of the L_0 norm—determinism controls were applied to improve reproducibility of the training trajectories and observed phase transitions.

A global random seed of 42 was applied across all computational backends before model initialization and data-loader construction. The following environmental settings were enforced in PyTorch to

disable non-deterministic GPU optimizations:

```
import os
import random
import numpy as np
import torch

# Global Seed
seed = 42
random.seed(seed)
os.environ['PYTHONHASHSEED'] = str(seed)
np.random.seed(seed)

# PyTorch and CUDA Determinism
torch.manual_seed(seed)
torch.cuda.manual_seed(seed)
torch.cuda.manual_seed_all(seed)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

By explicitly setting `cudnn.deterministic = True` and disabling the `cudnn.benchmark` auto-tuner, the environment reduces nondeterministic variation in convolutional operations, weight updates, and stochastic gate sampling. These controls make the reported trajectories reproducible within the specified software and hardware environment, while acknowledging that exact bitwise identity may not be guaranteed across different GPU architectures or library versions.

B Appendix B: Hyperparameter Configurations and Optimization Schedules

To support reproducibility and traceability of the empirical results and phase transitions documented in this study, the hyperparameter configurations, learning rate schedules, and optimization constraints for all four methodological phases are detailed below.

B.1 Hyperparameter Selection Rationale

The hyperparameters were chosen to balance three constraints: preserving the pretrained VGG19 representation, inducing measurable compression, and keeping validation performance within the study’s interpretive guardrails. Baseline fine-tuning used SGD with Nesterov momentum (Nesterov, 1983) because it provided stable transfer learning without rapidly overwriting pretrained ImageNet features. The baseline learning rate of 1×10^{-3} was large enough to adapt the classifier and upper convolutional features, while the step-decay schedule reduced the update size after early convergence.

For L_1 regularization, the learning rate was reduced to 5×10^{-4} because the objective included both cross-entropy and a growing sparsity penalty, following the Lasso principle of shrinkage and variable selection (Tibshirani, 1996). The λ schedule from 1×10^{-5} to 3×10^{-3} was selected as a gradual stress test: small early values allowed the model to retain its baseline decision rule, while larger later values revealed the point where sparsity began to damage class separation. The base hard threshold of 1×10^{-3} was chosen because the post- L_1 weight distribution showed a large near-zero mass below this level and a smaller tail of weights above it. Fully connected layers received a larger penalty because PCA showed stronger redundancy there; early convolutional layers received a smaller penalty to protect low-level image features.

For SVD, the energy-threshold grid $[0.10, 0.20, 0.30, 0.40, 0.50]$ was chosen to span aggressive through conservative low-rank compression, consistent with low-rank approximation theory (Eckart & Young, 1936; Mirsky, 1960). The selected threshold, 0.20, was the most aggressive setting that still preserved validation performance above the study guardrail before fine-tuning. Adam with learning rate 2×10^{-4} was used only for the newly factorized dense layers because the convolutional base was frozen and the new low-rank factors needed fast but stable adaptation (Kingma & Ba, 2015).

For L_0 Gaussian gating, separate learning rates were used because the pretrained convolutional weights and gate parameters had different roles, following differentiable sparse-learning work on L_0 regularization (Louizos, Welling, & Kingma, 2018). The convolutional weights used a conservative rate of 1×10^{-5} to avoid destroying pretrained features, while the gate means used 2×10^{-3} so channel-selection decisions could move quickly. The initial gate values $(\mu, \sigma) = (0.5, 0.5)$ kept all channels active at the start while placing gates near the open/closed decision boundary. The PID target of 60% gate sparsity was chosen as an aggressive but recoverable structural-compression target, and the λ cap limited collapse when validation F1 began to degrade, following the feedback-control logic of proportional–integral–derivative regulation (Åström & Hägglund, 1995).

B.2 Baseline VGG19 Training

The uncompressed baseline model was fine-tuned using Stochastic Gradient Descent (SGD) with Nesterov momentum. A standard step-decay learning rate scheduler (**StepLR**) was used to ensure stable convergence on the BloodMNIST+ dataset.

Table 19: Baseline optimization parameters.

Hyperparameter	Value	Description
Optimizer	SGD	Stochastic Gradient Descent
Epochs	15	Total training epochs
Base Learning Rate (η)	1×10^{-3}	Initial learning rate
Momentum	0.9	Nesterov momentum factor
Step Size	7	Epochs before learning rate decay
Gamma (γ)	0.1	Multiplicative factor of learning rate decay
Batch Size	64	Images per forward pass
AUC Guardrail	99.0%	Minimum validation AUC required to save checkpoint

The baseline settings in Table 19 were deliberately conservative: the learning-rate decay and AUC guardrail prioritize stable transfer learning over aggressive adaptation, making the uncompressed model a reliable reference for later compression.

Table 20: Baseline VGG19 training dynamics across 15 epochs. The model was trained with SGD and a step-decay learning rate schedule, with the best validation macro-F1 score of 98.86% reached at epoch 5.

Epoch	Learning Rate	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)	Val F1 (%)	Val AUC (%)	Min-Class F1 (%)	Status
1	0.00100	0.4729	0.1295	83.49	95.27	94.70	99.89	89.2	Best validation
2	0.00100	0.1229	0.0680	95.74	97.37	97.25	99.96	92.9	Best validation
3	0.00100	0.0962	0.0509	96.66	98.07	97.91	99.98	95.6	Best validation
4	0.00100	0.0662	0.0595	97.70	97.84	97.73	99.97	95.3	
5	0.00100	0.0590	0.0395	98.05	98.83	98.86	99.98	97.2	Best validation
6	0.00100	0.0532	0.0530	98.29	98.36	98.26	99.98	94.9	
7	0.00100	0.0400	0.0322	98.65	98.60	98.48	99.99	97.0	
8	0.00010	0.0289	0.0335	99.12	98.19	98.17	99.99	95.6	Step decay applied
9	0.00010	0.0255	0.0274	99.14	98.66	98.76	99.99	96.7	
10	0.00010	0.0241	0.0280	99.20	98.71	98.80	99.99	96.9	
11	0.00010	0.0228	0.0280	99.28	98.54	98.61	99.99	96.5	
12	0.00010	0.0224	0.0296	99.26	98.42	98.43	99.99	96.2	
13	0.00010	0.0205	0.0278	99.36	98.60	98.68	99.99	96.7	
14	0.00010	0.0204	0.0313	99.29	98.48	98.50	99.99	96.3	
15	0.00001	0.0168	0.0297	99.40	98.48	98.50	99.99	96.3	Final epoch

The baseline training log shows rapid convergence by epoch 5, after which validation macro-F1 fluctuated within a narrow range. This supports using the epoch-5 checkpoint as a stable high-performance reference rather than continuing to optimize for marginal training-loss reductions.

B.3 L_1 Lasso Regularization and IMP

The L_1 regularization phase used SGD with Cosine Annealing Warm Restarts (SGDR; Loshchilov & Hutter, 2017) to dislodge parameters hovering near the pruning threshold. Following the L_1 training phase, three cycles of Iterative Magnitude Pruning (IMP) and masked refinement were applied.

Table 21: L_1 Lasso optimization parameters.

Hyperparameter	Value	Description
Optimizer	SGD	Stochastic Gradient Descent
Epochs	30	Maximum L_1 training epochs; early stopped at 22
Base Learning Rate (η)	5×10^{-4}	Initial learning rate
Momentum	0.9	Nesterov momentum factor
SGDR Interval (T_0)	10	Epochs per cosine-annealing restart cycle
λ Schedule	$1 \times 10^{-5} \rightarrow 3 \times 10^{-3}$	Linear scaling of the L_1 penalty over 30 epochs
Base Hard Threshold	1×10^{-3}	Absolute magnitude cutoff for zeroing weights
FC λ Multiplier	$5.0\times$	Aggressive penalty for highly redundant dense layers
Early Conv λ Multiplier	$0.5\times$	Conservative penalty to protect low-level feature extractors

Table 21 defines L_1 training as a controlled stress test: the penalty rises gradually, fully connected layers receive stronger shrinkage, and early convolutional filters are protected with a smaller multiplier. This design tests redundancy without immediately damaging low-level feature extraction.

Table 22: L_1 Lasso regularization training dynamics. The λ penalty was scaled linearly from 1×10^{-5} to 3×10^{-3} . Training was early-stopped at epoch 22 after the validation macro-F1 score fell below the 88.0% clinical floor, indicating a phase transition where the regularization pressure exceeded the network’s representational capacity.

Epoch	λ Penalty	Train Loss	Val Loss	Val F1 (%)	Val AUC (%)	Min-Class F1 (%)	Soft Sparsity	Status
1	1.1×10^{-4}	293.678	0.041	98.10	99.99	96.2	21.5%	
2	2.1×10^{-4}	480.190	0.053	97.89	99.99	94.8	33.5%	
3	3.1×10^{-4}	553.001	0.026	98.96	99.99	97.1	47.9%	
4	4.1×10^{-4}	533.485	0.033	98.54	99.99	96.2	61.0%	
5	5.1×10^{-4}	474.656	0.036	98.42	99.99	96.3	70.5%	
6	6.1×10^{-4}	420.339	0.040	98.53	99.99	96.7	76.4%	
7	7.1×10^{-4}	388.568	0.043	98.83	99.99	97.0	79.7%	
8	8.1×10^{-4}	380.523	0.046	98.60	99.99	96.7	81.4%	
9	9.1×10^{-4}	392.500	0.051	98.51	99.98	96.7	82.1%	
10	1.0×10^{-3}	420.376	0.048	98.58	99.99	96.9	82.3%	
11	1.1×10^{-3}	357.756	0.104	97.07	99.94	92.5	87.1%	

Tables 22 and its continuation show the sparsity phase transition directly. Soft sparsity rose above 92% while validation F1 remained near baseline through epoch 20, but epochs 21–22 show that additional penalty pressure began to destroy class separation.

The IMP schedule uses two conservative refinement cycles before a stronger final threshold. This makes the pruning path interpretable: most sparsity is obtained before refinement, while later cycles test whether additional hard zeroing can be absorbed without destabilizing validation performance.

The IMP refinement table shows that hard sparsity stayed near 92.5–92.8% while validation F1 repeatedly recovered above 98%. This indicates that the zeroed weights were mostly redundant for prediction, although the model remained physically dense unless sparse storage or masking rules were imposed.

Table 23: L_1 Lasso regularization training dynamics (continued).

Epoch	λ Penalty	Train Loss	Val Loss	Val F1 (%)	Val AUC (%)	Min-Class F1 (%)	Soft Sparsity	Status
12	1.2×10^{-3}	273.892	0.093	96.91	99.95	92.2	89.0%	
13	1.3×10^{-3}	245.505	0.102	96.70	99.92	89.6	90.0%	
14	1.4×10^{-3}	234.700	0.077	97.94	99.95	95.0	90.8%	
15	1.5×10^{-3}	229.124	0.075	97.83	99.94	94.3	91.4%	
16	1.6×10^{-3}	226.206	0.150	93.65	99.94	82.2	91.9%	
17	1.7×10^{-3}	226.097	0.076	97.79	99.96	95.1	92.2%	
18	1.8×10^{-3}	229.137	0.084	97.56	99.95	95.1	92.4%	
19	1.9×10^{-3}	235.558	0.068	98.22	99.96	96.3	92.5%	
20	2.0×10^{-3}	244.958	0.066	98.19	99.97	95.9	92.5%	Best Checkpoint
21	2.1×10^{-3}	241.532	0.272	91.47	99.63	76.8	93.7%	
22	2.2×10^{-3}	211.897	0.513	72.23	98.90	23.2	94.7%	Phase Collapse (Stop)

Table 24: Iterative Magnitude Pruning (IMP) refinement schedule.

Cycle	Threshold Multiplier	Refinement Epochs	Refinement Learning Rate
Cycle 1	$1.0\times$	5	1×10^{-4} (Cosine Annealing)
Cycle 2	$1.0\times$	5	1×10^{-4} (Cosine Annealing)
Cycle 3	$1.5\times$	3	1×10^{-4} (Cosine Annealing)

Table 25: Iterative Magnitude Pruning (IMP) and masked refinement. Following the L_1 phase, three cycles of explicit tensor surgery (zeroing) and masked refinement (freezing zeroed gradients) were applied to translate soft sparsity into hard sparsity.

Cycle/Step	Phase	Threshold Multiplier	Val F1 (%)	Val Acc (%)	Val AUC (%)	Hard Sparsity
Cycle 1	Post-Surgery	$1.0\times$	98.25	98.13	99.97	92.5%
Cycle 1	Refinement Ep 1		97.93	97.84	99.96	92.5%
Cycle 1	Refinement Ep 2		98.04	98.19	99.96	92.5%
Cycle 1	Refinement Ep 3		98.27	98.19	99.97	92.5%
Cycle 1	Refinement Ep 4		97.58	97.78	99.95	92.5%
Cycle 1	Refinement Ep 5		98.52	98.48	99.97	92.5%
Cycle 2	Post-Surgery	$1.0\times$	98.52	98.48	99.97	92.5%
Cycle 2	Refinement Ep 1		97.96	98.13	99.96	92.5%
Cycle 2	Refinement Ep 2		97.93	97.90	99.97	92.5%
Cycle 2	Refinement Ep 3		98.22	98.36	99.97	92.5%
Cycle 2	Refinement Ep 4		97.97	97.96	99.97	92.5%
Cycle 2	Refinement Ep 5		98.26	98.31	99.97	92.5%
Cycle 3	Post-Surgery	$1.5\times$	98.14	98.19	99.97	92.8%
Cycle 3	Refinement Ep 1		98.51	98.48	99.97	92.8%
Cycle 3	Refinement Ep 2		97.81	97.90	99.95	92.8%
Cycle 3	Refinement Ep 3		98.32	98.31	99.98	92.8%

B.4 Singular Value Decomposition (SVD)

The SVD phase bypassed parameter-level pruning in favor of matrix factorization. The energy-threshold sweep was conducted on the validation set, followed by a brief joint adaptation phase in which the convolutional base was frozen and only the newly instantiated low-rank classification layers were updated.

Table 26: SVD factorization and fine-tuning parameters.

Hyperparameter	Value	Description
Energy Threshold Sweep	[0.10, 0.20, 0.30, 0.40, 0.50]	Target variance-retention limits evaluated
Selected Threshold	0.20	Optimal threshold preserving clinical guardrails
Optimizer	Adam	Adaptive Moment Estimation
Fine-Tune Epochs	10	Total epochs for joint adaptation
Fine-Tune Learning Rate	2×10^{-4}	Constant learning rate for low-rank layers
Pre-FT F1 Slack	8.0%	Allowable accuracy drop before fine-tuning recovery

The SVD parameter table records a deliberately narrow adaptation phase: the convolutional feature extractor is left unchanged, so any recovery after factorization must come from the low-rank classifier head. This isolates the effect of dense-layer low-rank structure.

Table 27: SVD low-rank joint fine-tuning dynamics over 10 epochs. The convolutional base was frozen, and the Adam optimizer with initial learning rate 2×10^{-4} was restricted to the newly factorized classification head.

Epoch	Train Loss	Val F1 (%)	Val AUC (%)	Min-Class F1 (%)	Learning Rate	Status
1	0.0988	98.50	99.99	96.5	2.00×10^{-4}	Best Val
2	0.0529	98.47	99.99	96.5	1.96×10^{-4}	
3	0.0461	98.29	99.99	95.9	1.83×10^{-4}	
4	0.0428	98.78	99.99	97.0	1.63×10^{-4}	Best Val
5	0.0434	98.56	99.99	96.5	1.38×10^{-4}	
6	0.0368	98.59	99.99	96.3	1.10×10^{-4}	
7	0.0365	98.47	99.99	96.3	8.22×10^{-5}	Best Val (Final)
8	0.0398	98.86	99.99	96.9	5.71×10^{-5}	
9	0.0381	98.63	99.99	96.5	3.72×10^{-5}	
10	0.0346	98.56	99.99	96.3	2.44×10^{-5}	

The SVD fine-tuning log shows that performance recovered almost immediately and peaked at epoch 8. This rapid recovery supports the interpretation that the discarded singular directions were mostly low-value redundancy rather than essential class information.

B.5 L_0 Gaussian Gates and PID Controller

The L_0 structured pruning phase required the most complex optimization environment. A dual-learning-rate Adam optimizer was used to decouple the training dynamics of the standard convolutional weights from the stochastic gate parameters (μ). A custom PID-style feedback controller dynamically adjusted the λ penalty to stabilize the network at a specific sparsity target.

Table 28: L_0 Gaussian gate optimization parameters.

Hyperparameter	Value	Description
Optimizer	Adam	Adaptive Moment Estimation
Epochs	40	Total training epochs
Weight Learning Rate	1×10^{-5}	Conservative rate for pretrained convolutional weights
Gate Learning Rate	2×10^{-3}	Aggressive rate for stochastic gate parameters (μ)
Gate Initialization (μ, σ)	0.5, 0.5	Places all gates at the open/closed decision boundary
λ Schedule	$1 \times 10^{-3} \rightarrow 1.5 \times 10^{-3}$	Linear base ramp, subject to PID adjustments
λ Hard Cap	2×10^{-3}	Maximum allowable penalty to prevent network collapse

Table 28 shows why the L_0 experiment behaves differently from L_1 and SVD: slow updates protect pretrained convolutional filters, while faster gate updates allow channel-selection decisions to move toward open or closed states. The hard cap on λ limits collapse when sparsity pressure becomes too strong.

Table 29: L_0 Gaussian gate PID-controlled training dynamics over 40 epochs. The controller targeted 60% gate sparsity and dynamically adjusted the λ penalty based on real-time channel survival. Epochs 1–10 are reported here.

Epoch	λ Penalty	Train Loss	Val Loss	Val F1 (%)	Val AUC (%)	Gate Sparsity	Weight Sparsity	Status
1	1.0×10^{-3}	5.051	0.182	95.5	99.9	0.0%	0.0%	Below Target
2	1.1×10^{-3}	4.781	0.454	94.7	99.9	6.0%	7.3%	Below Target
3	1.1×10^{-3}	4.597	0.211	97.5	99.9	12.8%	15.3%	Peak F1
4	1.1×10^{-3}	4.410	0.350	96.8	99.9	19.6%	23.3%	Below Target
5	1.2×10^{-3}	4.274	0.398	96.8	99.9	26.1%	30.7%	Below Target
6	1.2×10^{-3}	4.088	0.480	96.9	99.9	32.1%	37.4%	Below Target
7	1.2×10^{-3}	3.956	0.514	95.9	99.8	37.8%	43.6%	Below Target
8	1.3×10^{-3}	3.846	0.574	94.9	99.8	42.8%	48.9%	Below Target
9	1.3×10^{-3}	3.716	0.809	93.7	99.5	46.9%	53.1%	Below Target
10	1.3×10^{-3}	3.659	0.652	92.3	99.8	50.8%	57.0%	Below Target

The four L_0 epoch tables show the controller’s intended behavior. Sparsity increases quickly until the 60% target is crossed, then the penalty backs off and the model stabilizes near 69–70% gate sparsity; epoch 30 is selected because it balances this high sparsity with acceptable validation F1.

The PID rule table explains why the sparsity trajectory does not simply increase monotonically under a fixed penalty. Once the target is reached, the controller reduces regularization pressure, allowing the remaining channels to recover predictive performance instead of continuing toward uncontrolled pruning.

Table 30: L_0 Gaussian gate PID-controlled training dynamics (continued). Epochs 11–20.

Epoch	λ Penalty	Train Loss	Val Loss	Val F1 (%)	Val AUC (%)	Gate Sparsity	Weight Sparsity	Status
11	1.4×10^{-3}	3.575	0.963	89.7	99.4	53.9%	60.0%	Below Target
12	1.4×10^{-3}	3.523	0.349	94.4	99.7	56.6%	62.7%	Below Target
13	1.4×10^{-3}	3.415	0.582	90.9	99.5	59.1%	65.3%	Below Target
14	1.0×10^{-3}	3.318	0.353	93.0	99.7	60.9%	67.0%	Target Reached
15	1.0×10^{-3}	2.374	0.229	95.4	99.8	61.9%	68.0%	
16	1.0×10^{-3}	2.317	0.326	91.9	99.7	63.0%	69.2%	
17	1.0×10^{-3}	2.295	0.466	88.3	99.6	63.8%	69.9%	
18	1.0×10^{-3}	2.227	0.422	91.2	99.6	64.6%	70.7%	
19	1.0×10^{-3}	2.186	0.279	93.0	99.8	65.4%	71.4%	
20	1.0×10^{-3}	2.182	0.367	91.1	99.7	66.0%	71.8%	

Table 31: L_0 Gaussian gate PID-controlled training dynamics (continued). Epochs 21–30.

Epoch	λ Penalty	Train Loss	Val Loss	Val F1 (%)	Val AUC (%)	Gate Sparsity	Weight Sparsity	Status
21	1.0×10^{-3}	2.155	0.476	88.2	99.5	66.5%	72.3%	
22	1.0×10^{-3}	2.113	0.328	90.5	99.6	66.9%	72.7%	
23	1.0×10^{-3}	2.105	0.246	92.6	99.7	67.2%	72.9%	
24	1.0×10^{-3}	2.072	0.277	92.1	99.7	67.5%	73.2%	
25	1.0×10^{-3}	2.060	0.213	93.0	99.7	67.9%	73.5%	
26	1.0×10^{-3}	2.034	0.243	93.1	99.7	68.2%	73.7%	
27	1.0×10^{-3}	2.011	0.208	93.4	99.8	68.4%	74.0%	
28	1.0×10^{-3}	2.010	0.220	92.8	99.7	68.7%	74.2%	
29	1.0×10^{-3}	1.993	0.253	92.6	99.7	68.9%	74.4%	
30	1.0×10^{-3}	1.997	0.242	92.9	99.7	69.1%	74.5%	Best Checkpoint

Table 32: L_0 Gaussian gate PID-controlled training dynamics (continued). Epochs 31–40.

Epoch	λ Penalty	Train Loss	Val Loss	Val F1 (%)	Val AUC (%)	Gate Sparsity	Weight Sparsity	Status
31	1.0×10^{-3}	1.980	0.278	91.5	99.7	69.2%	74.7%	
32	1.0×10^{-3}	1.973	0.274	91.5	99.6	69.3%	74.8%	
33	1.0×10^{-3}	1.953	0.282	91.2	99.6	69.3%	74.8%	
34	1.0×10^{-3}	1.953	0.252	92.1	99.7	69.5%	74.9%	
35	1.0×10^{-3}	1.959	0.254	91.5	99.7	69.5%	75.0%	
36	1.0×10^{-3}	1.959	0.232	92.0	99.7	69.6%	75.0%	
37	1.0×10^{-3}	1.956	0.224	92.0	99.7	69.6%	75.0%	
38	1.0×10^{-3}	1.942	0.222	92.3	99.7	69.6%	75.0%	
39	1.0×10^{-3}	1.964	0.222	92.2	99.7	69.6%	75.0%	
40	1.0×10^{-3}	1.956	0.222	92.2	99.7	69.6%	75.0%	

Table 33: PID sparsity controller dynamics.

PID Condition	Multiplicative Action	Description
Target Sparsity	60.0%	Desired percentage of closed convolutional channels
Error > 5.0% (Sparsity << Target)	$\lambda_{\text{new}} = \lambda \times 1.03$	Aggressive penalty ramp (<code>pid_up_large</code>)
$0 < \text{Error} \leq 5.0\%$	$\lambda_{\text{new}} = \lambda \times 1.01$	Gentle penalty ramp (<code>pid_up_small</code>)
Error ≤ 0 (Sparsity $\geq$ Target)	$\lambda_{\text{new}} = \lambda \times 0.70$	Aggressive pressure release (<code>pid_backoff</code>)

C Appendix C: Dense Reconstruction via Tensor Surgery

This appendix describes the algorithm used to translate the theoretical “ghost sparsity” of the L_0 Gaussian gates into a physically smaller dense network. The goal is not to document a specific programming language, but to state the tensor operations required to remove dead channels while preserving the original input–output mapping of the masked model.

The algorithm scans the trained masked model, identifies the surviving convolutional channels using the learned gate means ($\mu > 0$), slices the corresponding convolutional tensors, and then remaps the flattened convolutional output into the first fully connected layer. This final remapping step is essential: after channels are removed from the last convolutional layer, the flattened feature vector is shorter, so the first dense layer must keep only the weight columns corresponding to surviving spatial-channel positions.

C.1 Tensor Surgery Procedure

Let M_{mask} denote the trained masked model, and let g_l denote the learned gate vector for convolutional layer l . For each layer, define the surviving output-channel set as

$$S_l = \{j : \mu_{l,j} > 0\}, \quad (\text{C.1})$$

where $\mu_{l,j}$ is the learned gate-location parameter for channel j in layer l .

The physical reconstruction proceeds as follows:

1. Initialize the surviving input-channel set for the first convolutional layer as $S_0 = \{1, 2, 3\}$, corresponding to the three RGB input channels.
2. For each convolutional layer l , read the surviving output channels S_l from the learned gate vector.
3. Construct a new convolutional layer with $|S_{l-1}|$ input channels and $|S_l|$ output channels, preserving the original kernel size, stride, padding, and activation order.
4. Copy only the active convolutional weights and biases:

$$W'_l = W_l[S_l, S_{l-1}, :, :], \quad b'_l = b_l[S_l]. \quad (\text{C.2})$$

Here, W_l and b_l are the original masked-model convolutional weight and bias tensors, while W'_l and b'_l are the physically reconstructed tensors.

5. Propagate the surviving output set forward by setting $S_{l-1} \leftarrow S_l$ before processing the next convolutional layer.
6. Copy non-parametric layers, such as ReLU and max-pooling, without changing their definitions.

7. After the final convolutional layer, build the flattened-index set for the first fully connected layer. If the final feature map has spatial size $H \times W$, then each surviving channel $c \in S_L$ contributes the index block

$$I_c = \{cHW, cHW + 1, \dots, cHW + HW - 1\}. \quad (\text{C.3})$$

The full retained flattened-index set is $I = \bigcup_{c \in S_L} I_c$.

8. Replace the first fully connected layer with a smaller layer whose input dimension is $|S_L|HW$, and copy only the retained columns of the original dense weight matrix:

$$A'_1 = A_1[:, I], \quad a'_1 = a_1, \quad (\text{C.4})$$

where A_1 and a_1 are the original first dense-layer weights and biases.

9. Copy the remaining fully connected layers unchanged, because their input and output dimensions are not affected after the first dense-layer remapping.
10. Assemble the sliced convolutional base and remapped classifier into the dense pruned model M_{dense} .

C.2 Numerical Consistency Check

To check whether tensor surgery changed the physical representation without materially changing the learned function, the masked model and dense pruned model were evaluated on the same diagnostic input tensor. The maximum absolute difference between their output logits was 1.9×10^{-6} , which is within floating-point precision. This provides numerical evidence that the reconstructed dense model preserves the masked model's mapping for the evaluated input while removing inactive channels from the executed architecture.